Lecture 5 — Actualization, Clean Architecture & OO Principles

> **Lecture id:** L05 **Source decks:** `Actualization.pdf` (28 slides; Rajlich Ch. 8), `Clean Architecture.pdf` (20 slides; Jan Sørensen), `OOPrinciples.pdf` (30 slides; Jan Sørensen — SOLID + GRASP) **Labs:** `ActualizationLab.pdf` (1 page; Jan Corfixen Sørensen) — cite as `[ActLab]` **Process phase(s):** Actualization (the implementation phase of the software change process) **Citation key:** `(Actualization p.X)`, `(Clean Architecture p.X)`, `(OOPrinciples p.X)` refer to the lecture decks by slide/page number; `[ActLab]` refers to the Actualization Lab handout. Reading keys: `[Raj13]` = Rajlich, *Software Engineering: The Current Practice* (Ch. 8 is the Actualization chapter); `[MC09]` = Martin & Coplien, *Clean Code*; `[GHJV94]` = Gamma/Helm/Johnson/Vlissides (Gang of Four), *Design Patterns*; `[Fowler99]` = Fowler, *Refactoring*; `[Martin]` = Robert C. Martin (the "Clean Code"/"Clean Architecture" body of work, incl. the SOLID principles and the 2000 paper "Design Principles and Design Patterns"); `[Larman04]` = Larman, *Applying UML and Patterns*, 3rd Ed., Prentice-Hall, 2004 (source of GRASP, cited on `(OOPrinciples p.17)`). **Grounding note:** Every slide of all three decks plus the 1-page lab was read in full (text layer + page images, since both the Clean Architecture and OOPrinciples decks store most content inside images). The Actualization deck is a near-verbatim Rajlich Ch. 8 deck. The Clean Architecture and OOPrinciples decks are the instructor's (Jan Sørensen) supplementary material — they are *not* Rajlich; treat their definitions as `[Martin]` / `[Larman04]`-sourced. Where I completed text that was clipped off the right edge of the Clean Architecture "Characteristics" slide `(Clean Architecture p.4)`, I used the well-known Robert C. Martin canonical wording and flagged it. No claim here is invented: everything traces to a specific slide listed in the Source Map.

Overview

This lecture covers the **Actualization** phase of Rajlich's software change process — the phase where programmers *actually write the new code* — together with two bodies of design knowledge that tell you *how to write that new code well*: **Clean Architecture** (Robert C. Martin's layered, dependency-rule architecture) and **Object-Oriented Principles** (the **SOLID** five, several supporting principles, and the nine **GRASP** patterns).

The change process is: **Initiation** → **Concept Location** → **Impact Analysis** → **Prefactoring** → **Actualization** → **Postfactoring** → **Conclusion**, all wrapped in continuous **Verification** `(Actualization p.1)`. Actualization sits in the middle: by the time you reach it, you already know *what* to change (Concept Location found the starting class), *where* the change will spread (Impact Analysis predicted the impact set), and the code has been *prefactored* so the new code has a clean place to be plugged in. Actualization is where you "implement the new functionality according to the change request" `(Actualization p.1)`.

Three ideas dominate Actualization itself:

1. **The size of the change dictates the technique** — small changes are edited directly into old code; larger changes are written as new classes, then **incorporated** (plugged in) `(Actualization p.1, p.2, p.5)`.
2. **Change propagation** — once new code is plugged in, secondary modifications **ripple** through the dependent classes until the system is consistent again; propagation is "the moment of truth" that confirms

or refutes Impact Analysis `(Actualization p.5, p.17–23)`.
3. **Impact is routinely under-estimated** — the Ericsson case study shows programmers correctly predicted *which* classes change with 100% precision but caught only 32% of them (recall), missing two-thirds `(Actualization p.24–28)`.

Clean Architecture and the OO principles are the *quality lens* on actualization: the lab explicitly asks you to "Understand and explain Clean Architecture in context of Actualization" and "Understand and explain Clean Code Principles in context of Actualization", and to give SOLID and Clean Architecture examples *from the CASE study* `[ActLab]`. The pedagogical thread is: good architecture + SOLID/GRASP make the new code easy to incorporate and limit how far change propagation ripples.

How the four sources divide the material

Knowing *which deck owns which claim* matters both for citations in the portfolio and for predicting question style:

- `Actualization.pdf` **(28 slides)** is the **Rajlich deck** — every slide carries the footer "Software Engineering: The Current Practice Ch. 8", and the incorporation-example slide additionally shows "© 2012 Václav Rajlich" `(Actualization p.16)`. It supplies the *process* content: the seven-phase diagram with the vertical VERIFICATION bar `(Actualization p.1)`, small vs. larger changes `(Actualization p.2–5)`, polymorphism as an actualization mechanism `(Actualization p.6–7)`, adding a new component and the three incorporation shapes `(Actualization p.8–11)`, the Point-of-Sale cashier change `(Actualization p.12–14)`, replacement of a class `(Actualization p.15)`, the colour-animated propagation walk `(Actualization p.16–21)`, deletion of obsolete functionality `(Actualization p.22)`, and the impact-accuracy block with the Ericsson numbers `(Actualization p.23–28)`. Exam questions sourced here tend to be *define / enumerate / compute*.
- `Clean Architecture.pdf` **(20 slides)** is **Jan Sørensen's deck** on Robert C. Martin's architecture: design pattern vs. architecture `(Clean Architecture p.2)`, the four success characteristics `(Clean Architecture p.3–4)`, the concentric-circle model with Martin's photo on the title slide of the section `(Clean Architecture p.5–8)`, the Boundary/Interactor data-flow Figures 1.1–1.6 `(Clean Architecture p.9–15)`, the database-is-a-detail block with the Martin plug-in quote and Figure 1.7 `(Clean Architecture p.16–18)`, the fully assembled Figure 1.8 `(Clean Architecture p.19)`, and a references slide of seven web sources `(Clean Architecture p.20)`. Questions sourced here tend to be *draw / label / explain-the-flow*.
- `OOPrinciples.pdf` **(30 slides)** is **Jan Sørensen's principles deck**: a chalkboard-styled title slide `(OOPrinciples p.1)`, SOLID history `(OOPrinciples p.2)` and the five one-line definitions `(OOPrinciples p.3)`, each SOLID principle as a definition slide + a "Without/With" example slide `(OOPrinciples p.4–13)`, CRP `(OOPrinciples p.14)`, PLK `(OOPrinciples p.15)`, the GRASP intro `(OOPrinciples p.16)` and Larman reference `(OOPrinciples p.17)`, the numbered list of nine patterns `(OOPrinciples p.18)`, one slide per pattern `(OOPrinciples p.19–27)`, and three blank trailing slides `(OOPrinciples p.28–30)`. Questions sourced here tend to be *match-principle-to-scenario* and *spot-the-violation*.
- `ActualizationLab.pdf` **(1 page)**, by Jan Corfixen Sørensen, ties the three together: it restates the actualization definition, sets two objectives (explain Clean Architecture and Clean Code Principles *in context of Actualization*), and demands portfolio work — SOLID examples and a Clean Architecture explanation *in context of the CASE study* `[ActLab]`. This is the synthesis layer: every "why does clean design make maintenance cheaper?" essay question is the lab restated.

Learning Objectives

After this lecture you should be able to:

1. **Define the Actualization phase** and state where it sits in the seven-phase change process, and explain that its content (implementing + incorporating + propagating) varies with change size `(Actualization p.1)`; `[ActLab]`.
2. **Distinguish small from larger changes** and choose the right technique: editing old code directly vs. writing new classes and incorporating them `(Actualization p.2–5, p.8)`.
3. **Explain incorporation** (plugging new code into existing code) and the three structural shapes of incorporation: new responsibility *local*, new responsibility *composite*, and *incorporating a new supplier* `(Actualization p.5, p.8–11)`.
4. **Explain polymorphism as an actualization mechanism** — extending a composite responsibility by adding a polymorphic subclass `(Actualization p.6–7)`.
5. **Trace change propagation** step by step through a dependency graph and state when it ends (the system is consistent again) `(Actualization p.16–22)`.
6. **Compute and interpret precision and recall** of impact analysis from a confusion matrix (true/false positives/negatives) and explain why under-estimation is the norm `(Actualization p.23–28)`.
7. **State the four characteristics of a successful (Clean) architecture**: testable, independent of UI, independent of database, independent of frameworks/external entities `(Clean Architecture p.3–4)`.
8. **Draw and explain the Clean Architecture concentric layers** (Entities → Use Cases → Interface Adapters → Frameworks & Drivers) and the **Dependency Rule** (source-code dependencies point only inward) `(Clean Architecture p.5–8)`.
9. **Explain the request/response data flow** through Boundary interfaces, the Interactor, Request/Response Models, Controller, Presenter and View Model `(Clean Architecture p.9–15)`.
10. **Argue that the database (and the UI, and frameworks) is a "detail" / plug-in**, and explain the Entity Gateway pattern that keeps it so `(Clean Architecture p.16–18)`.
11. **Define each SOLID principle** (SRP, OCP, LSP, ISP, DIP) and give the before/after code shape for each `(OOPrinciples p.3–13)`.
12. **Define the supporting principles**: Composite Reuse Principle (favor composition over inheritance) and Principle of Least Knowledge / Law of Demeter `(OOPrinciples p.14–15)`.
13. **Name and define the nine GRASP patterns** and what they are for (responsibility assignment) `(OOPrinciples p.16, p.18–27)`.
14. **Connect all of the above to the change process and to JHotDraw / the CASE study** — i.e. argue *why* clean design makes actualization cheaper and propagation shorter `[ActLab]`.

Key Concepts

Concept 1 — Actualization (the phase) and how it scales with change size

What it is. Actualization is the change-process phase in which *programmers implement the new functionality according to the change request* `(Actualization p.1)`. The lab gives the fullest definition: "The actualization phase consists of the **implementation of the new functionality**, its **incorporation into the old code**, and **change propagation** that seeks out and updates all places in the old code that require secondary modification." `[ActLab]`. So actualization = *implement* + *incorporate* + *propagate*. It is the only

phase in which production source code is actually written or rewritten; every other phase either decides *what* to change (Initiation, Concept Location, Impact Analysis) or *tidies* the result (Pre/Postfactoring, Conclusion).

What it's for / why it matters. Actualization is where the change request finally becomes running behaviour, so it is the phase that most directly determines whether a maintenance task is cheap or expensive. The reason the phase is named (rather than just "coding") is that *writing the code is only one of three jobs*: you must also slot it into the existing system and chase down the secondary edits it forces. Treating it as "just typing the feature" is exactly how maintainers blow their estimates — the incorporation and propagation work is usually larger than the implementation work. Framing actualization as three sub-activities gives the maintainer a checklist and gives the manager a basis for effort estimation.

When & how it's applied. You enter actualization once Impact Analysis has produced a predicted impact set and Prefactoring has opened a clean insertion seam. Concretely: you write the new logic, plug it in at the seam, then walk the dependency graph fixing everything that the plug-in made inconsistent. For a one-field tweak (the ZIP example below) all three sub-activities collapse into a single localized edit; for a feature like "add a cashier login" they are three visibly separate steps (write `Cashiers`, wire it into the launch path, update every client that assumed no authentication).

The process varies with the size of the change `(Actualization p.1)`. This is the central organising idea of the whole Actualization deck:

- **Small changes** are done *directly in the old code* `(Actualization p.2–4)`. The deck's running example is an `Address` class whose ZIP field is widened from `zip[5]` to `zip[9]` (5-digit → 9-digit ZIP) — a localized edit inside one class, no new structure `(Actualization p.2 → p.3, p.4)`. No incorporation, minimal propagation.
- **Larger changes**: "Programmers implement the new classes **separately** from the old code … The new code is **plugged into** the existing code (**incorporation**). The change can **propagate** to other components of the system (**ripple effect**)." `(Actualization p.5)`. New structure is built in isolation, then connected.

Why build separately first? Writing the new classes apart from the old code lets you get them right (and testable) before wiring them in; only then do you incorporate, which is the point at which ripple effects begin. This mirrors the change process: prefactoring (Lecture 4) prepared a clean insertion point; actualization fills it.

Anchor to the change process. Actualization is preceded by *Impact Analysis*, whose predicted impact set is exactly the set of classes you now expect to touch during incorporation and propagation; it is followed by *Postfactoring* (clean up the now-working code) and *Conclusion*, all under *Verification* `(Actualization p.1)`.

Concept 2 — Incorporation (plugging new code into old code)

What it is. Incorporation is the act of plugging newly written classes into the existing system so the old code begins to use them `(Actualization p.5, p.8)`. "New classes are plugged as **components** into the appropriate place of the existing code (**incorporation**)." `(Actualization p.8)`. The new classes "assume the responsibilities demanded by the change request" `(Actualization p.8)`. It is the connective step between *implementing* a feature in isolation and having it *exercised* by the real system — the moment the new class stops being an island and becomes wired into the existing dependency graph.

What it's for / why it matters. Incorporation exists because the recommended way to build a larger change is to write the new classes *separately* (so they can be developed and unit-tested cleanly) and only then

connect them. Incorporation is that connection step. It matters because the *place* and *shape* of the connection decide how far the change will then ripple: a leaf that nobody calls back into ripples almost nowhere, whereas a new *supplier* that many existing clients must now consult can trigger wide propagation. Choosing a good incorporation point (ideally the clean seam left by Prefactoring) is therefore the single biggest lever a maintainer has over the cost of the rest of the phase.

When & how it's applied. Whenever a change is "larger" — i.e. it introduces new structure rather than editing one field — you incorporate. In practice you identify the existing class that should now use the new code, add the dependency edge, and adjust that client so it actually invokes the new behaviour. The cashier-login worked example below is the canonical case: a freshly written `Cashiers` class is added to the model and then plugged into the launch path so the system can no longer start without a login.

The deck shows **three structural shapes** of incorporation (old-code box vs. new-code box, with arrows = dependencies); each tells you *how much* propagation to expect:

1. **New responsibility is local** `(Actualization p.9)` — the new code is a self-contained leaf: old code calls into it, but the new code calls nothing back out into the old system. This is the simplest and cheapest case because the dependency arrow runs one way only, so there is almost nothing to propagate. Use it whenever the feature can be expressed as a pure add-on the old code merely *invokes* (e.g. a new validation routine the old flow calls and otherwise ignores).
2. **New responsibility is composite** `(Actualization p.10)` — the new code itself depends on, or is composed of, further components, so incorporating it brings in a small *subtree* rather than a single node. This matters because the cost of incorporation now includes wiring up the new class's own collaborators; you are plugging in a cluster. It is the typical shape when the feature is a small subsystem (e.g. an authentication module that itself needs a credential store and a hasher).
3. **Incorporating a new supplier** `(Actualization p.11)` — the new class becomes a *supplier* (a dependency) that existing **client** classes now point to. This is the propagation-heavy case and the one to watch: because several existing clients now depend on the new supplier, *each* of them may need adjusting to use it, so this shape is where the ripple effect is most likely to spread widely. It is also the shape most relevant to Impact Analysis recall — the easiest clients to forget are the indirect ones that depend on the supplier transitively.

Replacement of a class `(Actualization p.15)` is a related operation: a new class takes over an old class's role; all references must be rerouted, and **deletion of the obsolete class** then triggers its own propagation (see Concept 5).

Worked incorporation (Point-of-Sale, see JHotDraw/CASE section). Old PoS had no authorization — "anyone was able to launch it" — and the change request was *"Create a cashier login that will control the user log in with a username and password."* `(Actualization p.12)`. A new `Cashiers` class is added to the model (`Store`, `Inventory`, `Item`, `Price`) `(Actualization p.13)` and then *incorporated* into the structure `(Actualization p.14)`. This is the canonical "add a new component and plug it in" case.

Concept 3 — Polymorphism as an actualization mechanism

What it is. Polymorphism lets you **extend a composite responsibility by adding a new subclass** that implements an existing operation differently, without editing the clients `(Actualization p.6–7)`. Mechanically it relies on inheritance plus *dynamic dispatch*: clients hold a base-class (or interface) reference and call a base-class operation; at run time the call is routed to whichever concrete subtype the object actually is. The new behaviour is "incorporated" not by re-wiring callers but simply by the existence of the new subclass.

What it's for / why it matters. It is the *least-rippling* technique available during actualization, which is exactly why the deck treats it as a first-class actualization mechanism rather than a mere language feature. Because existing clients keep talking to the unchanged base-class interface, adding a behaviour means adding code, not editing code — so change propagation is essentially confined to the new subclass itself. This is the design problem polymorphism solves: how to grow a system's behaviour without disturbing (and therefore without risking regressions in) the code that already works.

When & how it's applied. Reach for it whenever a new requirement is "one more variant of an existing family of things." The trigger phrase is *"behaviour varies by type."* You add a leaf subclass that overrides the family's operation; clients are untouched. This is the same shape as the Open/Closed Principle (Concept 13) and GRASP *Polymorphism* (Concept 19), and in JHotDraw it is the recipe for "add a new `Figure` subtype."

Deck example. A `Farm` holds `FarmAnimal`s; subclasses `Cow`, `Sheep`, `Pig` `(Actualization p.6)`. Adding `Pig` is pure new code:

```
class Pig : public FarmAnimal {
public:
    void makeSound() { cout << "Oink"; }
};
```

After this, "Farm now can declare objects of the type Cow, Sheep, or **Pig** — the **composite responsibility** of Farm was **extended** by the concept Pig." `(Actualization p.7)`. The new behavior is incorporated by virtue of inheritance + dynamic dispatch; `Farm` code is untouched.

Why this matters for maintenance. Polymorphic extension is the *least-rippling* form of actualization: you add a leaf subclass, and existing clients keep working through the base-class interface. This is exactly the Open/Closed Principle in action (Concept 9) and GRASP *Polymorphism* (Concept 17) — extend behavior by adding a type, don't modify a conditional.

Concept 4 — Change propagation and the ripple effect

What it is. Change propagation is the process that, after the new code is incorporated, *seeks out and updates every place in the old code that requires a secondary modification* `[ActLab]` `(Actualization p.5)`. A change to one class forces changes in classes that depend on it; those in turn force further changes — the **ripple effect** `(Actualization p.5)`. Where incorporation is the *primary* modification (the deliberate plug-in point), propagation is the cascade of *secondary* modifications that the plug-in makes necessary to keep the system internally consistent.

What it's for / why it matters. Propagation is not an optional clean-up step — it is what makes the change actually correct. After incorporation, some classes have inconsistent assumptions (they were written against the old behaviour); leaving them un-updated yields a system that compiles but is wrong. Propagation matters for management too: it is the work that Impact Analysis tried to predict, and its actual extent is the "moment of truth" that scores that prediction. The whole point of low coupling and clean architecture (later concepts) is to *bound* this propagation so it terminates quickly.

When & how it's applied. It runs immediately after incorporation and is performed as a graph walk: starting from the modified class, you visit each dependent, decide whether it is now inconsistent, fix it if so, and then visit *its* dependents — continuing until no inconsistent class remains. Crucially it also applies to deletions: removing obsolete code forces you to chase down and delete every reference, which propagates just like an addition.

Mechanics (the propagation walk). The Actualization deck animates a dependency graph of `sale`, `register`, `saleLineItem`, `store`, `item`, `taxCategory` over several slides `(Actualization p.16–21)`: - A class is modified (the **incorporation** point) `(Actualization p.16)`. - Its dependents become *inconsistent* and must be visited and updated; the "modified frontier" advances class by class `(Actualization p.17 → p.18 → p.19 → p.20)`. - **Change propagation ends** when no inconsistent classes remain — the system is internally consistent again `(Actualization p.21)`. Termination is the goal: a propagation that doesn't converge signals a design problem.

Deletion also propagates. "Deletion of obsolete functionality also causes change propagation. **All references** to the deleted functionality must be deleted; secondary changes propagate to other classes." `(Actualization p.22)`. Removing code is not free — it ripples just like adding code.

Propagation = the moment of truth. "Impact analysis estimates which classes are impacted. Change propagation modifies the code of impacted classes — change propagation is **the moment of truth**: it confirms or refutes the predictions of impact analysis." `(Actualization p.23)`. The accuracy of those predictions "is important for software managers" because it drives effort estimates `(Actualization p.23)`.

Concept 5 — Measuring impact-analysis accuracy: precision & recall (Ericsson)

What it is. A way to *quantify* how good the earlier Impact Analysis prediction was, by treating "which classes will change?" as an information-retrieval problem: the predicted-changed set is the "retrieved" set, the actually-changed set is the "relevant" set, and you score the overlap with **precision** and **recall**. When propagation finishes you can compare *predicted* vs. *actual* changed classes and score the prediction `(Actualization p.23–27)`.

What it's for / why it matters. These two numbers convert a vague feeling ("we underestimated") into a measured, comparable figure that managers can use for effort estimation and process improvement. They separate two distinct failure modes: raising false alarms (low precision) versus missing real work (low recall). The Ericsson study's punchline is that the second failure mode dominates in practice — programmers rarely flag classes that turn out *not* to change, but they routinely miss classes that *do* — so recall, not precision, is the metric that wrecks schedules.

When & how it's applied. You apply it after change propagation has finished, when the *actual* changed set is finally known. You tabulate predicted vs. actual into a confusion matrix, read off TP/FP/TN/FN, and compute the two ratios. The same procedure scores a JHotDraw change just as it scored the Ericsson telecom code.

The Ericsson Radio Systems confusion matrix `(Actualization p.24)` (rows = predicted, columns = actual; total classes = 42 + 0 + 64 + 30 = **136**):

	Actual: Unchanged	**Actual: Changed**
Predicted Unchanged	42	64
Predicted Changed	0	30

Categories `(Actualization p.25)`: - **True positives = 30** (predicted changed *and* changed) - **False positives = 0** (predicted changed but did *not* change) - **True negatives = 42** (predicted unchanged and unchanged) - **False negatives = 64** (predicted unchanged but *did* change — the dangerous ones)

Precision `(Actualization p.26)` — of the classes you *predicted* would change, what fraction actually did:

> Precision = TP / (TP + FP) = 30 / (30 + 0) = 1 = **100%**.

Perfect precision means *no false alarms*: every class flagged really did need changing.

Recall `(Actualization p.27)` — of the classes that *actually* changed, what fraction did you predict:

> Recall = TP / (TP + FN) = 30 / (30 + 64) = 0.32 = **32%**.

"Programmers estimated that the changes will impact only about a **third** of all classes that actually changed — **missed the other two thirds!**" `(Actualization p.27)`.

Under-estimation is systemic `(Actualization p.28)`: it is "common in software engineering, a **consequence of invisibility**" (you cannot see all dependencies), it "makes planning difficult", and it is "common in other fields also." Exam takeaway: high precision is easy; high recall is hard, and the cost of low recall is blown schedules.

Concept 6 — Clean Architecture: what it is and why it differs from a design pattern

What it is. A **software architecture** "refers to the high-level structures of a software system and the discipline of creating such structures and systems" — examples given are **Hexagonal architecture, Onion architecture, and The Clean architecture** `(Clean Architecture p.2)`. A **design pattern**, by contrast, "is the re-usable form of a solution to a design problem" — examples: **Singleton, Observer, MVC, MVVM, MVP** `(Clean Architecture p.2)`. The two operate at different scales: a pattern is a small, reusable template that solves one recurring *local* problem inside a few classes, whereas an architecture is a system-wide scheme for how the *whole* codebase is partitioned and how those partitions may depend on one another. `[GHJV94]` is the pattern reference; Clean Architecture is `[Martin]`.

What it's for / why it matters. The distinction is worth labouring because it is a classic exam trap and because the two answer different questions. Patterns answer "how should these particular objects collaborate?"; architecture answers "what are the system's layers, and which way are dependencies allowed to point?" **The Clean Architecture** specifically — Robert C. Martin's architecture (credited on the slide as "Robert C. Martin (Clean Code)") `(Clean Architecture p.5)` — exists to insulate a system's business rules from all volatile technical concerns (UI, database, frameworks). Its purpose, in maintenance terms, is to make a whole *category* of future change requests cheap: if the volatile thing is on the outside and the stable rules are on the inside, "change the volatile thing" never reaches the rules.

When & how it's applied. You apply Clean Architecture when designing or restructuring a system you expect to outlive its current UI toolkit, database, or web framework. You partition code into concentric layers (Concept 8), keep all business logic in the inner layers, and push every framework, driver, and I/O concern to the outer layer behind interfaces. In this lecture it is applied as the *quality lens on actualization*: it is the structural reason a well-designed change ripples only through the outer layers.

Concept 7 — The four characteristics of a successful architecture

What it is. A checklist of four independence properties that, per Martin, characterise a successful software architecture `(Clean Architecture p.3)`: - **Testable** - **Independent of the UI** - **Independent of the Database** - **Independent of Frameworks and External Entities**

What it's for / why it matters. Each property is a deliberate hedge against one *category* of future change. They matter because the whole value proposition of Clean Architecture is captured here: an architecture that has these four properties keeps its business rules stable while everything fragile around them changes. Testability is the keystone — if the business rules can be tested without spinning up a UI, a database, or a framework, then by construction they do not depend on any of those things, and the other three independences follow.

When & how it's applied. Treat the four as design acceptance criteria. When you finish a design, ask: can I unit-test the rules with no UI/DB/framework? Can I re-skin the UI without touching the rules? Can I swap the database without touching the rules? Can I replace a framework without rewriting business logic? Below, each is expanded with the slide's wording and the *kind of change request* it neutralises.

Expanded `(Clean Architecture p.4)`: - **Independent of frameworks.** "Libraries and frameworks should be *used as tools*, rather than having to cram your system into their constraints." (right edge clipped on slide; canonical `[Martin]` completion). - **Testable.** "Business rules have tests that are independent of UI, Database, [and any external element]." - **Independent of UI.** "UI can change easily, without changing other system components." - **Independent of Database.** "The database can be switched from RDBS to NoSQL or any other [DB]" — the business rules don't care. - **Independent of any external agency.** "Business rules should know nothing about the outside world."

Connection. Each independence is a hedge against a *category of future change request*. If the UI is independent, a "change the UI" request ripples only through the outer layer; if the DB is independent, a "swap the database" request never reaches the entities. This is the architectural reason actualization stays cheap.

Concept 8 — Clean Architecture layers and the Dependency Rule

What it is. The structural heart of Clean Architecture: a set of concentric layers plus a single rule (the Dependency Rule) governing which way source-code dependencies may point. The layers partition responsibility by *stability* — innermost = most stable/abstract, outermost = most volatile/concrete — and the rule forbids any inner layer from knowing about any outer one.

What it's for / why it matters. This is the mechanism that *delivers* the four independences of Concept 7. By forcing all dependencies inward toward stable abstractions, the architecture guarantees that volatile outer details (frameworks, DB, UI) can be replaced without disturbing the inner business rules — replacements happen "below the line." For maintenance this is decisive: it is the structural reason change propagation during actualization stays bounded to the outer rings.

When & how it's applied. When placing a new class, ask which ring it belongs in (Does it hold enterprise rules? application rules? does it adapt formats? is it a framework/driver?) and then ensure every dependency it introduces points *inward*. If an inner class seems to need an outer one, you invert the dependency: declare an interface in the inner layer and implement it in the outer layer (the Entity Gateway of Concept 10).

The Clean Architecture is drawn as **concentric circles** `(Clean Architecture p.5–8)`. From inner (most stable) to outer (most volatile):

1. **Entities** (innermost) — *What:* "enterprise-wide business rules that encapsulate the most general business rules; these rules are the least likely to change." `(Clean Architecture p.7)` (colour band: **Enterprise Business Rules**). These are the concepts and invariants that would be true of the business *even if this particular application did not exist* — the domain objects and rules an organisation reuses across many systems. *For:* holding the knowledge that changes least often, so it can be the bedrock everything else

depends on. *Example:* in JHotDraw, the `Drawing` / `Figure` / `Handle` model — the rules of "what a drawing is" — independent of any editor UI.

2. **Use Cases** — *What:* "also called *interactors*; **application-specific** business rules. This layer is isolated from changes to the database, common frameworks, and the UI." `(Clean Architecture p.7)` (colour band: **Application Business Rules**). A use case orchestrates the entities to accomplish one specific thing the *application* (not the wider enterprise) needs to do. *For:* encoding application workflow ("the dance of the entities") without binding it to any delivery technology. *Example:* a `Login` use case that validates a cashier's credentials and decides what happens next — the *steps*, independent of which screen or database is involved.
3. **Interface Adapters** — *What:* a translation layer that "convert[s] data from a convenient format for entities and use cases to a format applicable to databases and the web. This layer includes **Presenters** (from MVP), **ViewModel** (from MVVM), and **Gateways** (also known as **Repositories**)." `(Clean Architecture p.7)` (outer-ring labels: Controllers, Presenters, Gateways). *For:* shielding the inner layers from outer formats so neither side has to speak the other's language — the inner layers work in domain objects, the outer layers in JSON/SQL/pixels, and this ring converts between them. *Example:* a Presenter turning a `Response Model` `Date` object into a formatted string, or a Gateway turning a domain object into SQL rows.
4. **Frameworks and Drivers** (outermost) — *What:* "the outermost layer, which consists of the web framework, database, UI, HTTP client, and so on." `(Clean Architecture p.7)` (labels: Web, DB, Devices, UI, External Interfaces). This is where all the volatile, vendor-supplied, replaceable technology lives. *For:* keeping every concrete tool at the edge so it is a *detail* that plugs in (Concept 10) rather than a fixture the rest of the system is built around. *Example:* the Swing toolkit and the concrete SQL/NoSQL driver behind a JHotDraw-style app — the things you most expect to swap over the system's life.

The Dependency Rule `(Clean Architecture p.8)`. The diagram pairs the circles with a pyramid and a flow arrow: - The pyramid axis runs from **"Abstract, General, Rarely Change"** (Entities, top) down to **"Concrete, Specific, Change Frequently"** (External Interfaces, bottom) `(Clean Architecture p.8)`. - **Source-code dependencies point only inward.** The flow diagram on the same slide shows UI → Presenter → Use case → **Entity** at the centre, with the **"Dependency rule"** arrows pointing toward the Entity `(Clean Architecture p.8)`. Outer layers know about inner layers; **inner layers know nothing about outer layers**.

Why inward. Inner = stable abstractions; outer = volatile details. Depending inward means details depend on abstractions, never the reverse — this *is* the Dependency Inversion Principle applied at architecture scale (Concept 13). It guarantees the independences of Concept 7.

Concept 9 — Boundaries, Interactor, Request/Response Models (the data-flow mechanics)

What it is. The concrete object-level machinery that lets a request travel from the UI all the way to the entities and back *while still obeying the Dependency Rule*. Where Concept 8 gives the static layering, this concept gives the dynamic flow: the named objects (Boundary interfaces, Interactor, Request/Response Models, Controller, Presenter, View Model, View) and the order in which control and data pass through them.

What it's for / why it matters. The arrangement exists to solve a subtle problem: the outer UI must *trigger* the inner use case, yet source dependencies must point *inward*. Boundary interfaces resolve this — the interactor declares and implements an input boundary, so the UI depends on an abstraction owned by the inner layer, not the other way round. The Request/Response Models matter because they are *plain primitive data* with no framework types, which is what keeps the inner layers ignorant of the outside world. Getting this right is what makes the architecture's independences real rather than aspirational.

When & how it's applied. You apply this whenever you wire a delivery mechanism (web form, console, mobile UI) to a use case. The vocabulary the decks introduce `(Clean Architecture p.9)`:

- **Interactor (Use Case)** — "Contains application-specific business rules." `(Clean Architecture p.9)`. It "controls the dance of the entities" `(Clean Architecture p.11)` — i.e. it is the conductor that drives the entities through the steps of one use case. It is what you *call into* to run application logic.
- **Entity** — "Contains application-*independent* business rules." `(Clean Architecture p.9)`. The domain object the interactor manipulates; reusable beyond this one application.
- **Boundary** `<I>` — an interface/protocol that forms the *seam* between an interactor and the world. "Data comes into interactors through a boundary (interface/protocol) which is implemented by the interactor. Data goes out from the interactor to boundaries implemented by other objects." `(Clean Architecture p.9)`. Its job is to invert the dependency so the UI depends inward on an abstraction.
- **Delivery Mechanism** — the concrete way the use case is presented to a user. "Can be Web, Console application, Mobile App UI, etc." `(Clean Architecture p.9)`. It is deliberately interchangeable — one of the "details" the architecture isolates.

The request/response cycle `(Clean Architecture p.10–13)`: 1. User clicks a button on a web form `(Clean Architecture p.10)`. 2. The **delivery mechanism** packs the submitted data into a plain data structure of *primitive types only* — the **Request Model** `(Clean Architecture p.10)`. 3. The Request Model passes through the **input boundary**; since the interactor implements that boundary, the interactor receives the request model `(Clean Architecture p.10)`. 4. The interactor uses the request model and **controls the dance of the entities** (creates the order, modifies the customer, …) `(Clean Architecture p.11)`. 5. It gathers the results into another data structure, the **Response Model** `(Clean Architecture p.12)`. 6. The response model is passed back out through the **output boundary** to the delivery mechanism, which delivers it to the user `(Clean Architecture p.13)`.

Definitions of the outer objects `(Clean Architecture p.15)` — each carries the inner→outer translation a little further, deliberately *demoting* the data from rich domain objects to dumb display strings so the View can stay untestably trivial: - **Response Model** — *What:* "data that was created by the interactor as the result of the use case; this data is not presentable, a date for example would be a date object, and the currency would be a money object." *For:* carrying the use case's result outward as pure domain data with no presentation decisions baked in. *How:* the interactor fills it and hands it across the output boundary; it still speaks in `Date` and `Money` objects, not strings. - **Presenter** — *What:* "a class or a set of classes whose job is to take the response model and translate it into a view model … generates a nice date string that the view will hold, and takes the currency symbol in the view model." (this is the MVP "Presenter"). *For:* moving all *formatting* and presentation logic out of both the entities and the View, so it lives in one testable place. *How:* it consumes the Response Model's objects and produces display-ready primitives (a formatted date string, a currency symbol). - **View Model** — *What:* "contains the data that will be used by the view … the string that holds the name of that button, and if the button should/n't be active … the boolean that indicates the status of the button … menu items in the view, … the names of the menu items and their order." It holds *strings and flags*, ready to display. *For:* being a passive, fully-prepared data bag so the View has zero decisions left to make. *How:* the Presenter writes it; the View only reads it. - **View** — *What:* "all it can do is grabbing the data from the view model and put it on the screen. It should be so stupid so you don't have to test it." `(Clean Architecture p.15)`. *For:* keeping the genuinely untestable, framework-bound rendering layer as thin as humanly possible, so untested code carries no logic. *How:* it copies View Model fields onto widgets — no conditionals, no formatting, nothing worth a unit test.

The full assembled flow (Controller → Boundary → Interactor → Entities, and back via Response Model → Boundary → Presenter → View Model → View) is shown in Figure 1.6 `(Clean Architecture p.14)`.

Concept 10 — The database (and UI, frameworks) is a *detail* / plug-in; the Entity Gateway

What it is. A design stance plus the pattern that enforces it. The stance: the database (and by extension the UI and frameworks) is a *detail* — a replaceable plug-in — not the centre of the system. The pattern that makes the stance structurally real is the **Entity Gateway** (a.k.a. Repository): an interface declared *with* the business rules but implemented *out* in the detail layer.

What it's for / why it matters. This concept is the punchline of Clean Architecture for maintenance. If the DB is a plug-in behind a gateway interface, then a change request as drastic as "switch from SQL to NoSQL" is actualized *entirely* in the gateway implementation; change propagation physically cannot reach the entities or use cases because they depend only on the gateway *interface*, which is unchanged. The architecture thereby **bounds the ripple** — the single biggest reason clean design makes actualization cheaper. It also forbids the common anti-pattern of putting business logic in stored procedures, which would weld the rules to the database vendor.

When & how it's applied. Apply it whenever an inner layer needs to reach an outer resource (database, file system, remote API). Instead of the use case calling the database directly (which would point a dependency outward), you declare a gateway interface in the inner layer, have the use case depend on *that*, and write the concrete implementation below the boundary line where it may freely use the database API. This is Dependency Inversion (Concept 16) applied at architecture scale, and GRASP *Pure Fabrication* (Concept 19) supplies the made-up `PersistentStorage`-style class that holds it.

Thesis `(Clean Architecture p.17)`: - "The **database is a detail**, it shouldn't be the center of your architecture." - "The database should be a **plug-in** to the business rules." - "Business rules **shouldn't be written in stored procedures** in the database."

Quoted rationale from Martin `(Clean Architecture p.16)`:

> "If something **changes a lot**, it should be a **plug-in**. If something **doesn't change very often**, it should be **plugged into**." — Robert C. Martin

So volatile things (DB, UI, frameworks) plug into stable things (business rules), never the reverse.

The Entity Gateway pattern `(Clean Architecture p.18, Figure 1.7)`: the interactor depends on an `Entity Gateway <I>` interface; an `Entity Gateway Implementation` sits *below the boundary line* (outside the business rules) and talks to the `Database API`. The gateway interface is declared *with* the business rules, but implemented *out* in the detail layer — so the dependency still points inward (DIP). This is the **Repository** pattern named on `(Clean Architecture p.7)`. The complete picture combining the Presenter side and the Gateway side is Figure 1.8 `(Clean Architecture p.20 → references slide)`; the assembled diagram is shown on the page before References).

Connection to actualization. Because the DB is a plug-in behind a gateway interface, a change request like "switch from SQL to NoSQL" is actualized entirely in the *Entity Gateway Implementation* — change propagation cannot reach the entities or use cases. The architecture *bounds the ripple*.

Concept 11 — SOLID, the umbrella; history & one-line definitions

What it is. SOLID is a mnemonic for five complementary object-oriented design principles (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion) that together

describe how to structure classes and their dependencies so the code stays flexible. **History** `(OOPrinciples p.2)`: "80% of software projects fail. The theory of SOLID principles was introduced by **Robert C. Martin** in his **2000 paper 'Design Principles and Design Patterns'**. The **SOLID acronym itself was introduced later by Michael Feathers.**" `[Martin]`.

What it's for / why it matters. The five principles are not independent rules to memorise in isolation; they pull in the same direction — toward low coupling and code that absorbs change by *extension* rather than *modification*. That is precisely what makes a system maintainable, and why the lab asks you to find SOLID examples in the CASE study: a SOLID-conforming design is one where actualization ripples less and Impact-Analysis recall is easier to achieve. The "80% fail" framing on the slide is the motivation — undisciplined dependencies are a leading cause of systems that become too rigid to change.

When & how it's applied. You apply SOLID while writing the new classes during actualization (and while refactoring during Pre/Postfactoring). Each principle has a characteristic *violation smell* and a characteristic *fix*; the five concepts below pair each with the deck's before/after code so you can recognise and repair the smell.

The five, one line each `(OOPrinciples p.3)`: - **S**ingle Responsibility — "A class should have only a single responsibility (i.e. changes to only one part of the software's specification should be able to affect the specification of the class)." - **O**pen/Closed — "Software entities … should be open for extension, but closed for modification." - **L**iskov Substitution — "objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program." - **I**nterface Segregation — "many client-specific interfaces are better than one general-purpose interface." - **D**ependency Inversion — one should "depend upon abstractions, [not] concretions."

The next five concepts define each in detail with the deck's before/after code.

Concept 12 — Single Responsibility Principle (SRP)

What it is. "A class should have only **one reason to change.** — Robert C. Martin" `(OOPrinciples p.4)`; equivalently, a class should have a single responsibility `(OOPrinciples p.3)`. The operative phrase is *reason to change*: a "responsibility" here means an axis along which the requirements might evolve, and each class should sit on only one such axis. A class that two different stakeholders (say, the security team and the user-account team) would both want to edit is carrying two responsibilities.

What it's for / why it matters. SRP directly shrinks change propagation. If each class answers to one reason to change, a given change request touches few classes, those classes are easy to reason about in isolation, and there is little risk that editing one responsibility accidentally breaks an unrelated one tangled into the same class. In the lecture's terms it improves Impact-Analysis recall (fewer hidden couplings to miss) and keeps the ripple short during actualization.

When & how it's applied. Apply it whenever a class starts accumulating verbs from different domains. The fix is *extraction*: split the unrelated responsibility into its own class. The smell to watch for is a class whose name is vague or whose methods fall into two clearly different groups.

Deck example `(OOPrinciples p.5)`. *Without "S"* — `UserService` both changes passwords *and* verifies access (two reasons to change: password policy and access policy):

```
class UserService {
  void changePassword(User user) {
    if (checkAccess(user)) { /* Grant option to change */ }
```

```
  }
  boolean checkAccess(User user) { /* Verify if the user is valid. */ }
}
```

With "S" — access checking is split into its own `SecurityService`:

```
class UserService {
  void changePassword(User user) {
    if (SecurityService.checkAccess(user)) { /* Grant option to change */ }
  }
}
class SecurityService {
  static boolean checkAccess(User user) { /* check the access. */ }
}
```

Now password policy and access policy each have *one reason to change*.

Maintenance connection. SRP directly limits change propagation: if each class has one responsibility, a change request touches few classes, so Impact Analysis recall improves and ripple is short.

Concept 13 — Open/Closed Principle (OCP)

What it is. "Classes should be **open for extension but closed for modification**." `(OOPrinciples p.6)`. "OCP states that we should be able to **add new features** to our system **without having to modify** our set of pre-existing classes." `(OOPrinciples p.6)`. "One of the tenets of OCP is to **reduce the coupling** between classes to the abstract level — instead of creating relationships between two concrete classes, we create relationships between a concrete class and an abstract class or an interface." `(OOPrinciples p.6)`. The two words name the goal precisely: *open for extension* means new behaviour can be added; *closed for modification* means the existing, tested source files need not be reopened to do so.

What it's for / why it matters. OCP is the most exam-relevant bridge between "OO principles" and "actualization," because adding behaviour by *extension* rather than *modification* is exactly the least-rippling form of change (Concept 3). Every time you can satisfy a change request by adding a new class instead of editing old ones, you avoid the risk of regressions in working code and you confine change propagation to the new code. That is the whole maintainability payoff.

When & how it's applied. The trigger is a class that must keep growing to support new *variants* of something (new loan types, new figure shapes, new payment methods). The fix is to introduce an abstraction (interface/abstract class) the class depends on, and add each new variant as a fresh implementation of that abstraction. The smell is a `switch` / `if-else` chain on a type code, or a class that you keep editing every time a new variant appears.

Deck example `(OOPrinciples p.7)`. *Without "O"* — `LoanApprovalHandler.approve` is hard-wired to a concrete `PersonalLoanValidator`; supporting a new loan type means editing the handler. *With "O"* — introduce a `Validator` interface and let `PersonalLoanValidator` / `HomeLoanValidator` implement it; `LoanApprovalHandler` now depends on `Validator` and is *closed to modification but open to new validators*:

```
interface Validator { boolean isValid(); }
class PersonalLoanValidator implements Validator { boolean isValid() {} }
class HomeLoanValidator   implements Validator { boolean isValid() {} }
class LoanApprovalHandler {
  void approve(Validator validator) {
    if (validator.isValid()) { /* Process the loan. */ }
```

```
  }
}
```

Connection. OCP is the design embodiment of the polymorphic-extension actualization technique of Concept 3 (add a `Pig`, don't edit `Farm`). It is the single most exam-relevant link between "OO principles" and "actualization."

Concept 14 — Liskov Substitution Principle (LSP)

What it is. "**Subclasses should be substitutable for their base classes.**" `(OOPrinciples p.8)` — the **substitutability principle** `(OOPrinciples p.8)`. Per `(OOPrinciples p.3)`: "objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program." The key insight is that this is a *behavioural contract*, not merely a type-compatibility check: a subtype must honour everything callers are entitled to assume about the base type, not just compile in its place.

What it's for / why it matters. LSP is what makes polymorphism (Concept 3) and OCP (Concept 13) *safe*. Those techniques let clients work through a base-class reference without knowing the concrete subtype; that only works if every subtype actually behaves like a valid base type. If a subtype breaks the contract, then adding it during actualization *introduces* defects and *causes* propagation (clients must now special-case it) rather than absorbing the change cleanly — defeating the very point of polymorphic extension.

When & how it's applied. Apply LSP whenever you add a subclass. The smell is a subclass that overrides a method to throw "unsupported," to weaken a guarantee, or to demand more than the base contract — a sign the inheritance hierarchy is modelling "is-a" where no true behavioural is-a exists. The fix is to restructure the hierarchy so the unsupported capability lives only on the subtypes that genuinely have it.

Deck example `(OOPrinciples p.9)`. *Without "L"* — `Bird` has `fly()`; `Ostrich extends Bird` but `fly()` throws `UnsupportedOperationException()` — an Ostrich is *not* substitutable for a Bird, breaking any client that calls `fly()`:

```
class Bird { public void fly(){} public void eat(){} }
class Sparrow extends Bird {}
class Ostrich extends Bird { fly() { throw new UnsupportedOperationException(); } }
```

With "L" — restructure the hierarchy so only flight-capable birds expose `fly()`:

```
class Bird { void eat(){} }
class FlightBird extends Bird { void fly(){} }
class NonFlightBird extends Bird {}
class Sparrow extends FlightBird {}
class Ostrich extends NonFlightBird {}
```

Every subtype now honours its base type's contract.

Connection. LSP protects polymorphic actualization (Concept 3): if a newly added subclass violates the base contract, incorporating it *introduces* defects and *causes* propagation rather than absorbing it.

Concept 15 — Interface Segregation Principle (ISP)

What it is. "**Many specific interfaces are better than a single, general interface.**" `(OOPrinciples p.10)`. "Any interface we define should be **highly cohesive**." `(OOPrinciples p.10)`. Per `(OOPrinciples`

`p.3)`: "many client-specific interfaces are better than one general-purpose interface." Put differently: no client should be forced to depend on methods it does not use. An interface should be tailored to the *role* a particular client needs, not be a catch-all of every operation an implementer might offer.

What it's for / why it matters. A "fat" interface broadens coupling: every client and every implementer is bound to the *whole* interface, so a change to one method's signature ripples to all of them even if they never touch that method. Segregated, cohesive interfaces confine that ripple to the clients of the slice that actually changed — again shortening change propagation during actualization. It also keeps implementers honest: they no longer have to stub out methods they don't support (which is often a smell that LSP is about to be violated too).

When & how it's applied. Apply ISP when an interface accumulates methods serving unrelated clients. The smell is implementers with empty/throwing method bodies, or callers that only ever use a fraction of an interface. The fix is to split the fat interface along client-role lines into several cohesive interfaces, each one the minimal contract its client needs.

Deck example `(OOPrinciples p.11)`. *Without "I"* — one fat interface forces every implementer to carry methods it doesn't need:

```
interface IUser { changePassword(); checkUserRole(); assignRole(); }
```

With "I" — split into cohesive, client-specific interfaces:

```
interface IUser     { changePassword(); }
interface IRole     { assignRole(); }
interface IUserRole { checkUserRole(); }
```

Clients depend only on the slice they use.

Connection. Fat interfaces broaden coupling, so a change to one method's signature ripples to *all* implementers; segregated interfaces confine that ripple — again limiting change propagation during actualization.

Concept 16 — Dependency Inversion Principle (DIP)

What it is. "**Depend upon abstractions. Do not depend upon concretions.**" `(OOPrinciples p.12)`. "DIP formalizes the concept of abstract coupling and clearly states that we should **couple at the abstract level, not at the concrete level.**" `(OOPrinciples p.12)`. Crucially: "**DIP tells us how we can adhere to OCP.**" `(OOPrinciples p.12)`. The word *inversion* refers to flipping the conventional dependency direction: instead of high-level policy depending on low-level detail, both depend on a shared abstraction, so the detail effectively depends on the policy's interface.

What it's for / why it matters. DIP is the *mechanism* behind the two biggest ideas in this lecture. At class scale it is how you achieve OCP (depend on an abstraction and new concretions can be added without modification). At architecture scale it *is* the Clean Architecture Dependency Rule (Concept 8): details depend inward on stable abstractions, never the reverse — the Entity Gateway (Concept 10) is DIP made architectural. For maintenance, depending on abstractions means a class is insulated from changes to (and choices of) the concrete things it collaborates with, so swapping a concrete implementation does not ripple into it.

When & how it's applied. The smell is a class that `new`s up a concrete collaborator inside itself, hard-wiring the two together. The fix is to depend on an interface/abstract type and supply the concrete instance from outside — typically via **constructor injection** (note: this is the *principle*, achieved here with plain DI, not necessarily a DI framework).

Deck example `(OOPrinciples p.13)`. *Without "D"* — `Payments` `new`s a concrete `CreditCardPaymentMethod`, so it is welded to credit-card logic:

```
class Payments {
  CreditCardPaymentMethod ccpm = new CreditCardPaymentMethod();
  void makePayments() { ccpm.transact(); }
}
```

With "D" — depend on the `PaymentMethod` abstraction, injected via the constructor:

```
class Payments {
  PaymentMethod pm;
  Payments(PaymentMethod pm_provided) { pm = pm_provided; }
  void makePayments() { pm.transact(); }
}
```

This is constructor **dependency injection**; `Payments` no longer knows which concrete payment method it uses.

Connection. DIP *is* the Clean Architecture Dependency Rule at class scale (Concept 8): details depend on abstractions, dependencies point toward the stable abstraction. The Entity Gateway (Concept 10) is DIP made architectural.

Concept 17 — Composite Reuse Principle (CRP): favor composition over inheritance

What it is. "**Favor polymorphic composition of objects over inheritance.**" `(OOPrinciples p.14)`. "One of the most **catastrophic mistakes** that contribute to the demise of an object-oriented system is to use **inheritance as the primary reuse mechanism**." `(OOPrinciples p.14)`. "**Delegation** can be a better alternative to inheritance." `(OOPrinciples p.14)`. In plain terms: when one class needs the behaviour of another, prefer to *hold an instance of it and call it* (composition + delegation) over *subclassing it* to inherit the behaviour. Inheritance is a compile-time, statically-fixed "is-a" relationship; composition is a run-time "has-a" relationship whose collaborator can be chosen, swapped, or reconfigured.

What it's for / why it matters. The maintainability problem CRP solves is the *fragility of deep inheritance*. Inheritance exposes the parent's internals to the child and binds them permanently, so a change in a base class can break subclasses far down the tree (the "fragile base class" problem), and the structure is fixed at compile time. Composition localises change: because the collaborator is reached only through its public interface and held by reference, you can swap one delegate for another without touching the surrounding class, and behaviour can vary per object at run time. That flexibility is exactly what keeps actualization cheap when requirements shift the *combination* of behaviours rather than adding a clean new type.

When & how it's applied. Use composition/delegation when you are tempted to subclass merely to *reuse code* (rather than to model a true behavioural is-a). The smell is an inheritance hierarchy built for code sharing, or one that grows combinatorially (a subclass for every mix of features). The fix is to extract the reused behaviour into a collaborator object and delegate to it. *Caveat (see Pitfalls):* the slide condemns

inheritance as the **primary** reuse mechanism, not all inheritance — genuine is-a polymorphism (the `Farm` / `Pig` and OCP examples) still uses inheritance correctly. This pairs with the GoF maxim "favor object composition over class inheritance" `[GHJV94]`.

Concept 18 — Principle of Least Knowledge (PLK) / Law of Demeter

What it is. A coupling-limiting rule about *who an object is allowed to talk to*. "For an operation **O** on a class **C**, only operations on the following objects should be called: **itself, its parameters, objects it creates, or its contained instance objects.**" `(OOPrinciples p.15)`. "Also called the **Law of Demeter**." `(OOPrinciples p.15)`. "The basic idea is to **avoid calling any methods on an object where the reference to that object is obtained by calling a method on another object** (*transitive visibility*)." `(OOPrinciples p.15)`. In short: an object should converse only with its *immediate* collaborators (self, arguments, things it made, its own fields) and never "reach through" one object to reach another.

What it's for / why it matters. PLK exists to suppress *transitive* coupling — the indirect, hard-to-see dependencies that form when code navigates an object graph. The slide names the benefit directly: "the calling method doesn't need to understand the structural makeup of the object it's invoking methods upon." `(OOPrinciples p.15)`. This is profoundly relevant to this lecture: transitive, invisible dependencies are exactly the ones Impact Analysis fails to predict, and the explanation the deck gives for the Ericsson 32%-recall under-estimation is the *invisibility* of dependencies (Concept 5). By keeping each object ignorant of the internal structure of its collaborators, PLK ensures that a change inside one object's internals does not ripple into every method that happened to navigate through it.

When & how it's applied. The smell is a "train wreck" call chain like `a.getB().getC().doThing()` — three objects' internal structures leaking into one caller. The fix is to *tell, don't ask*: give the immediate collaborator a method that does the work, so the caller says `a.doThing()` and never sees `B` or `C`. GRASP *Indirection* (insert a mediator) and *Protected Variations* (wrap the unstable structure behind a stable interface), defined below, are the patterns that operationalise PLK.

Concept 19 — GRASP: the nine responsibility-assignment patterns

What it is. GRASP = "**General Responsibility Assignment Software Patterns.**" `(OOPrinciples p.16)`. "A set of **9 patterns** introduced as a learning aid by **Craig Larman**" `(OOPrinciples p.16)` `[Larman04]` `(OOPrinciples p.17)`. They "describe fundamental principles for object-oriented design and **responsibility assignment**, expressed as patterns." Each pattern answers one recurring question of OO design — *"which class should be given this responsibility?"* — and packages the reasoning as a named, reusable guideline rather than leaving the choice to intuition.

What it's for / why it matters. "The skillful assignment of responsibilities is extremely important in object design" `(OOPrinciples p.16)`, because *where* a responsibility lives determines coupling, cohesion, and ultimately how change ripples. Two of the nine (Low Coupling, High Cohesion) are even *evaluative* patterns — yardsticks you use to judge whether any particular assignment was good. GRASP matters for this lecture because clean responsibility assignment is what makes a design SOLID and Clean-Architecture-friendly, and therefore what keeps actualization's change propagation short.

When & how it's applied. "Determining the assignment of responsibilities often occurs during the creation of **interaction diagrams**, and certainly during programming." `(OOPrinciples p.16)`. In practice you apply them whenever you decide which class gets a new method during actualization — using the matching pattern as the rationale (who *has the data*? → Information Expert; who *contains* the thing? → Creator; who handles a

system event? → Controller; behaviour *varies by type*? → Polymorphism; need to *decouple* two parts? → Indirection; nowhere in the domain to put it? → Pure Fabrication; protecting against *future change*? → Protected Variations).

The nine `(OOPrinciples p.18)`: **1. Information Expert, 2. Creator, 3. Low Coupling, 4. High Cohesion, 5. Controller, 6. Polymorphism, 7. Indirection, 8. Pure Fabrication, 9. Protected Variations.** Each is defined below with *what it is*, *what it's for*, and *when/how to apply it*.

1. Information Expert `(OOPrinciples p.19)` — *What:* "As a general principle of assigning responsibilities to objects, assign a responsibility to the **information expert**: i.e. **the class that has the information necessary to fulfill the responsibility.**" *For:* keeping behaviour next to the data it needs, which minimises coupling (no fetching data from elsewhere) and maximises cohesion — the default, most-used GRASP pattern. *When/how:* ask "which class already knows the data required to compute this?" and put the method there. *Example:* to compute a sale's total, the `Sale` (which holds the line items) does it, not an external manager class that would have to reach in for the data.

2. Creator `(OOPrinciples p.20)` — *What:* "Assign class **B** the responsibility to **create** an instance of class **A** if one or more of the following is true: B **aggregates** A objects; B **contains** A objects; B **records** instances of A objects; B **closely uses** A objects; B **has the initializing data** that will be passed to A when it is created (thus B is an Expert with respect to creating A)." *For:* deciding *who instantiates whom* so that creation responsibility follows existing structural relationships rather than being scattered arbitrarily, which keeps coupling low. *When/how:* prefer the class that *aggregates or contains* A as A's creator. *Example:* a `Sale` creates its own `SaleLineItem`s because it contains them.

3. Low Coupling `(OOPrinciples p.21)` — *What:* an *evaluative* pattern — "Assign a responsibility so that **coupling remains low.** A class with high (or strong) coupling relies on many other classes; such classes may be undesirable — changes in related classes **force local changes**, they are **harder to understand in isolation**, and **harder to reuse** because their use requires the additional presence of the classes on which they depend." *For:* directly attacking the cause of wide change propagation — the fewer classes a class depends on, the fewer places a change can ripple to. *When/how:* among competing designs for an assignment, pick the one that introduces the fewest new dependencies. It is a tie-breaker you apply alongside the other patterns, not a rule about a single method.

4. High Cohesion `(OOPrinciples p.22)` — *What:* the other *evaluative* pattern — "Assign a responsibility so that **cohesion remains high.** A class with low cohesion does many unrelated things, or does too much work; such classes are undesirable — **hard to comprehend, hard to reuse, hard to maintain, and delicate (constantly affected by change).**" *For:* keeping each class focused on one coherent purpose, which is the class-level counterpart of SRP and makes classes easy to understand and stable under change. *When/how:* when an assignment would make a class do unrelated work, give the responsibility elsewhere (or fabricate a new class). High cohesion and low coupling are usually pursued together and reinforce each other.

5. Controller `(OOPrinciples p.23)` — *What:* "Assign the responsibility for **receiving or handling a system event message** to a class representing one of: the overall system/device/subsystem (a *façade controller*), or a *use-case / session controller* (a class representing a use-case scenario; use the same controller for all system events in one use-case scenario). Note: **window, applet, widget, view, and document classes are *not* on this list** — such classes should *not* fulfill system-event tasks; they should **delegate** these events to a controller." *For:* providing a single first point of contact between the UI and the domain logic, so that input handling does not leak business logic into UI classes (this is the GRASP justification for the Clean Architecture Controller/Interactor split). *When/how:* when a system event arrives

(button click, request), route it to a façade or use-case controller; have the view merely forward it. *Trap:* UI/view classes must **not** be the controller — a common exam pitfall.

6. Polymorphism `(OOPrinciples p.24)` — *What:* "When related alternatives or behaviors **vary by type (class)**, assign responsibility for the behavior — **using polymorphic operations** — to the types for which the behavior varies. Define the behavior in a common base class or, preferably, in an **interface**." *For:* replacing type-checking conditionals (`if type == X … else if type == Y …`) with dynamic dispatch, so new variants are added as new types rather than as new branches in existing code — the GRASP statement of OCP and of the actualization `Pig` example. *When/how:* when you see a conditional switching on an object's type, give each type its own polymorphic implementation of the operation. *Example:* JHotDraw's `Figure.draw()` overridden per shape, instead of one `draw()` that switches on a shape code.

7. Indirection `(OOPrinciples p.25)` — *What:* "Assign the responsibility to an **intermediate object** to mediate between other components or services so that they are **not directly coupled.** The intermediary creates an *indirection* between the other components. **Beware of transitive visibility.**" *For:* decoupling two parts that would otherwise depend on each other directly, by inserting a mediator/adapter in between — the structural means by which the Principle of Least Knowledge is honoured. *When/how:* when two components must collaborate but you want them independent (so either can change or be replaced), route their interaction through an intermediary. *Example:* a Gateway/adapter mediating between a use case and a database API so neither knows the other directly.

8. Pure Fabrication `(OOPrinciples p.26)` — *What:* "Assign a highly cohesive set of responsibilities to an **artificial or convenience class that does not represent a problem-domain concept** — something made up, to support high cohesion, low coupling, and reuse. Example: a class solely responsible for saving objects in a persistent storage medium such as a relational database; call it `PersistentStorage` — a *figment of the imagination*." *For:* giving a home to behaviour (like persistence, logging, or formatting) that does not naturally belong to any domain entity, so that forcing it onto a domain class would wreck that class's cohesion. *When/how:* when Information Expert would point you at a domain class but doing so would pollute it with non-domain concerns, invent a new non-domain class instead. *Connection:* this is exactly the Clean Architecture **Gateway/Repository** idea — keep persistence out of the entities by fabricating a storage class.

9. Protected Variations `(OOPrinciples p.27)` — *What:* "Identify **points of predicted variation or instability**; assign responsibilities to create a **stable interface around them.** Note: 'interface' is used in the broadest sense of an *access view*; it does not literally only mean a Java or COM interface." *For:* the grand unifying pattern — it isolates whatever is expected to change behind a stable seam so that change cannot ripple past the seam, which is the deepest reason clean design bounds change propagation. *When/how:* anticipate where requirements (or external dependencies) are likely to shift — a database vendor, a UI toolkit, a calculation rule — and put a stable interface in front of each. *Connection:* OCP, DIP, Indirection, and the Entity Gateway are all special cases of Protected Variations.

Concept 20 — The colour-coded propagation animation (the taxCategory walk, slides 16–21)

What it is. The Actualization deck's propagation example is not just a list of class names — it is a six-slide *animation* over a fixed class diagram, and the colours carry the algorithm. The diagram `(Actualization p.16)` : a top row `register` — `store` — `item` , a bottom row `sale` — `saleLineItem` , with edges `register` – `sale` , `item` – `saleLineItem` , and the **new class** `taxCategory` (drawn as a **blue** box) attached to `item` . The colour convention across the six slides: **blue** = the newly incorporated class; **green** = the class currently *marked for inspection* (it may be inconsistent); **red** = a class that *was modified* (a confirmed secondary change); **grey** = a class that was *inspected and needed no change* (or is done); **white** = not yet reached.

The walk, slide by slide:

1. `(Actualization p.16)` "Example incorporation" — `taxCategory` (blue) is plugged into the system; a thick grey arrow points from `taxCategory` into `item`, which is **green**: `item` is the **incorporation point** and the first class that must be inspected and adapted to use the new supplier. This is the only slide in the deck with the visible "© 2012 Václav Rajlich" credit.
2. `(Actualization p.17)` "Change propagation" — `item` has turned **red** (it was modified). Thick black arrows fan out from `item` to its dependency neighbours: `store` (now **green**, arrow pointing left into it) and `saleLineItem` (now **green**, arrow pointing down into it). Both must be inspected because they interact with the changed `item`.
3. `(Actualization p.18)` — `store` has turned **grey**: it was inspected and required *no* secondary change. The frontier continues at `saleLineItem`, still green with the arrow from `item`.
4. `(Actualization p.19)` — `saleLineItem` has turned **red** (modified). The thick arrow now runs left along the `sale` – `saleLineItem` edge; `sale` turns **green**.
5. `(Actualization p.20)` — `sale` has turned **red** (modified). The arrow runs up the `sale` – `register` edge; `register` turns **green**.
6. `(Actualization p.21)` "Change propagation ends" — `register` has turned **grey** (inspected, unchanged). No green class remains, so the walk terminates: the system is consistent again.

Final tally. Changed (red): `item`, `saleLineItem`, `sale`. Inspected-but-unchanged (grey): `store`, `register`. New (blue): `taxCategory`. Untouched: none — every class in this small graph was at least visited.

Why this matters for the exam. The animation encodes three facts examiners probe: (a) propagation travels along *dependency edges*, one neighbour at a time; (b) **visited ≠ changed** — `store` and `register` were inspected but stayed grey, so they are *not* secondary modifications; (c) termination is detected when the marked (green) set is empty `(Actualization p.21)`. The tally also feeds directly into the precision/recall machinery of `(Actualization p.24–27)`: the red set is the "actually changed" set you compare against the Impact-Analysis prediction.

Concept 21 — Reading the incorporation diagrams closely (UML detail of slides 9–11 and 15)

What it is. Slides 9, 10, 11, and 15 share one fixed visual vocabulary: a two-compartment frame labelled **"Old Code"** (left) and **"New Code"** (right); the old code is always the same five-class hierarchy — a top class holding two middle classes via UML **composition** (filled diamonds), each middle class holding one bottom class via composition; the new class is always drawn **grey**. What changes between the slides is *which edges cross the boundary*, and that is exactly what distinguishes the incorporation shapes:

- **New responsibility is local** `(Actualization p.9)` — the grey new class stands entirely **alone** in the New Code compartment: *no* edges are drawn to or from it. The visual point: the new responsibility is self-contained; the old five-class structure is not redrawn or re-wired at all. This is the cheapest shape — the old code will merely call into the newcomer.
- **New responsibility is composite** `(Actualization p.10)` — the grey class now has **two composition diamonds of its own** pointing across the boundary into the two *bottom-layer old classes*. Read precisely: the new class is a **composite whose parts are existing old-code classes** — incorporating it means reusing old suppliers as its components. The old hierarchy itself is still structurally unchanged.
- **Incorporating new supplier** `(Actualization p.11)` — two additions relative to p.10: the **top old class gains a composition diamond pointing across the boundary at the new class** (an old *client* now aggregates the new *supplier*), while the new class still aggregates the two bottom old classes.

The new class is wired both ways — used from above, using below — which is why this shape carries the largest propagation risk.
- **Replacement of a class** `(Actualization p.15)` — same layout as p.11, except one of the old *middle* classes is **crossed out with an X**: the new grey class takes over both its role (the top class now aggregates the new class instead) and its connections to the bottom classes. The crossed-out class is obsolete and must be deleted — and per `(Actualization p.22)` that deletion triggers its own propagation, because "all references to the deleted functionality must be deleted."

Connection to the Farm slide. The polymorphism diagram `(Actualization p.6)` uses the same Old Code / New Code frame: `Farm`, `Cow`, `Sheep`, and `FarmAnimal` sit in Old Code; the grey `Pig` box sits in New Code with a single **inheritance** (hollow-triangle generalization) edge running to `FarmAnimal`. Note the edge type: incorporation by *inheritance* (p.6) vs. incorporation by *composition* (p.9–11) — the deck deliberately shows both wiring mechanisms.

Concept 22 — The Point-of-Sale diagrams in close-up (slides 13–14)

What it is. The cashier-login change `(Actualization p.12)` is rendered in two diagram slides whose differences are the entire message:

- **PoS + new class** `(Actualization p.13)`: the new `Cashiers` class (dark grey) stands **unconnected** at the left, while the existing model forms a composition chain — `Store` ◆— `Inventory` ◆— `Item` ◆— `Price` (each holder marked with a filled composition diamond). At this stage the new class is implemented but still an island: written *separately from the old code* exactly as `(Actualization p.5, p.8)` prescribes.
- **Incorporation of Cashier** `(Actualization p.14)`: one new association now links `Store` to `Cashiers`, with the composition diamond on the **Store** side — `Store` aggregates `Cashiers`. `Store` itself is shaded **light grey**, marking it as the *modified* old class (the incorporation point). `Inventory`, `Item`, and `Price` are untouched white boxes.

The visual lesson. Incorporation, at minimum, costs *one new edge plus one modified old class*. Whether `Inventory` / `Item` / `Price` ever get touched is then a question for change propagation, not incorporation. Structurally this is the "incorporating a new supplier" shape of `(Actualization p.11)`: `Store` (old client) now depends on `Cashiers` (new supplier).

Concept 23 — The stability pyramid and the dependency-rule flow diagram (Clean Architecture p.8, both halves)

What it is. Slide 8 of the Clean Architecture deck pairs two diagrams that answer two different questions about the same architecture:

- **Left half — the pyramid ("Level").** A triangle sliced into four bands: **Entities** at the apex, **Use Cases** beneath, then a band containing **Controllers / Gateways / Presenters**, and at the base **Devices, Web, DB, External Interfaces, UI**. A vertical axis labelled **Level** runs alongside, annotated **"Abstract, General, Rarely Change"** at the top and **"Concrete, Specific, Change Frequently"** at the bottom `(Clean Architecture p.8)`. The colour legend (Enterprise Business Rules / Application Business Rules / Interface Adapters / Frameworks & Drivers) matches the circle diagram of `(Clean Architecture p.6)`. The pyramid answers: *which layer is more stable?* — higher = more abstract, more general, changes more rarely.

- **Right half — the flow diagram.** A dark diagram of nested rings with a **vertical dashed arrow labelled "Data Flow"** running top-to-bottom through **UI → Presenter → Use case → Entity** (the centre) and continuing down through **Repository → Data source**; separate **solid horizontal arrows labelled "Dependency rule"** point inward at the Entity `(Clean Architecture p.8)`. This answers: *which way do things move?* — and the answer is **two different ways**: *data* flows straight through every layer (from the UI all the way down to the data source and back), while *source-code dependencies* always aim at the centre.

The exam-critical distinction. Data flow and dependency direction are **not the same arrow**. A request's data passes outward-in *and* inward-out during one use case, but no inner-layer source file ever names an outer-layer type. The Boundary interfaces and the Entity Gateway exist precisely to let data cross a boundary *against* the direction that dependencies are allowed to point (Concepts 9–10). If an exam answer says "dependencies follow the data flow," it is wrong — that is the single most common misreading of this slide.

Concept 24 — Figure-by-figure: the request/response cycle (Figures 1.1–1.6) and the gateway figures (1.7–1.8)

What it is. The Clean Architecture deck's mechanics section is organised as eight numbered figures. Knowing which figure shows what lets you cite (and redraw) precisely:

- **Figure 1.1** `(Clean Architecture p.9)` — the static cast. Left to right: the **User** (stick figure) ↔ **Delivery Mechanism** (tall box); a **double vertical line** (the boundary between delivery mechanism and application); two **Boundary** `<I>` interface boxes; the **Interactor**; three **Entity** boxes fanned out to the right. The slide's bullets define each role: Interactor = "Contains application specific business rules"; Entity = "Contains application independent business rules"; Boundary `<I>` = "Data comes into interactors through boundary (interface/protocol) which is implemented by the interactor. Data goes out from the interactor to boundaries implemented by other objects"; Delivery Mechanism = "Can be Web, Console application, Mobile App UI, etc".
- **Figure 1.2** `(Clean Architecture p.10)` — steps 1–3 of the cycle. A red **Request Model** ellipse appears: "**1-** User clicks a button (on a web form for example). **2-** The delivery mechanism (ex: Web) takes whatever data submitted by the user and stick it into a data structure which contains only primitives types (Request Model). **3-** The request model passed through the input boundary and since the interactor implements the input boundary, the interactor received that request model."
- **Figure 1.3** `(Clean Architecture p.11)` — step 4. Red dashed arrows run from the Interactor out to the three Entities: "**4-** Interactor then uses that request model and controls the dance of the entities (ex: creates the order, modify the customer, etc)."
- **Figure 1.4** `(Clean Architecture p.12)` — step 5. Blue dashed arrows run from the Entities back to the Interactor, and a blue **Response Model** ellipse appears beneath it: "**5-** And when it's done with that, it continues to control the dance of the entities but this time in the opposite direction as it gather up all the results to create yet another data structure called (Response Model)."
- **Figure 1.5** `(Clean Architecture p.13)` — step 6. A blue arrow carries the Response Model through the output boundary back into the Delivery Mechanism: "**6-** The response model is passed back out through the output boundary to the delivery mechanism that implement it and somehow it's delivered to the user."
- **Figure 1.6** `(Clean Architecture p.14)` — the cycle redrawn with the Delivery Mechanism split into its real parts: **Controller** → input **Boundary** `<I>` (implemented by the **Interactor**); the Interactor produces the **Response Model** (blue path) which exits via the output **Boundary** `<I>` to the **Presenter**; the Presenter writes the **View Model**; the **View** reads the View Model. The red path (Controller →

Request Model → Interactor) is the request; the blue path (Interactor → Response Model → Presenter) is the response. The double vertical line still separates delivery mechanism (left) from application (right).

- **Figure 1.7** `(Clean Architecture p.18)` — the persistence side. The **Interactor** points down to an **Entity Gateway** `<I>` interface; below a **horizontal double boundary line**, the **Entity Gateway Implementation** realizes that interface (hollow-triangle realization arrow pointing *up* across the line) and points to the **Database API**. The three Entities sit above the line. The realization arrow crossing the line upward *is* the Dependency Inversion Principle drawn as architecture.
- **Figure 1.8** `(Clean Architecture p.19)` — everything assembled in one picture: Controller, Presenter, View Model, View on the left of the vertical boundary; Boundary interfaces, Request/Response Models on the line; Interactor, Entities, and Entity Gateway `<I>` in the application; Entity Gateway Implementation and Database API below the horizontal boundary. This is the one diagram worth practising to redraw from memory — it contains the whole lecture's Clean Architecture content in a single frame.

Concept 25 — Fine print worth marks: Creator's tie-break, Controller's session vocabulary, and the View joke

What it is. Three small slide details that summaries usually drop but that distinguish a complete exam answer:

- **Creator's tie-break rule.** The Creator slide closes with: "If more than one option applies, prefer a class B which **aggregates** or **contains** class A." `(OOPrinciples p.20)`. So when several classes qualify (one records A, another closely uses A, a third contains A), the *container/aggregator* wins. The slide also states the conclusion explicitly: "B is a *creator* of A objects."
- **Controller's two kinds and the session.** The Controller slide names the two admissible choices precisely: a class that "Represents the overall system, device, or subsystem (**facade controller**)", or one that "Represents a use case scenario within which the system event occurs (a **use-case- or session-controller**)" — with two sub-bullets: "Use the **same controller class** for all system events in the **same use case scenario**" and "Informally, a **session** is an instance of a conversation with an actor." `(OOPrinciples p.23)`. The negative list follows: "window," "applet," "widget," "view," and "document" classes are *not* on this list — "Such classes should *not* fulfill the tasks associated with system events, they typically **delegate** these events to a controller."
- **The View tested "with your eyes."** The View definition ends with a parenthetical the deck means literally: the View "should be so stupid so you don't have to test it. **(You can test it with your eyes.)**" `(Clean Architecture p.15)`. That is the deck's own formulation of the testability characteristic `(Clean Architecture p.3–4)`: push all logic out of the untestable rendering layer until visual inspection suffices.

JHotDraw Connection

The course's standard case study is **JHotDraw** (a Java GUI framework for structured drawing editors), used to ground every change-process phase. This lecture's decks instead use *smaller* worked examples (Address/ZIP, Farm/animals, Point-of-Sale, and a `sale` / `register` POS model), but they map onto JHotDraw and the CASE study directly, and the **lab makes the JHotDraw/CASE link mandatory**:

- **The lab assignment** `[ActLab]` requires you to: "Provide examples of the **SOLID principles in context of the CASE study**" and "Explain **Clean Architecture in context of the CASE Study.**" So in the exam/portfolio you are expected to point at concrete JHotDraw classes and name which SOLID principle / which Clean Architecture layer they exemplify.

- **OCP & Polymorphism in JHotDraw.** JHotDraw's `Figure` hierarchy (e.g. `RectangleFigure`, `EllipseFigure`, `LineFigure` implementing/extending a `Figure` abstraction) is the textbook OCP/GRASP-Polymorphism case: adding a new figure type is *new code plugged in*, with `DrawingView` / `DrawingEditor` clients untouched — the exact "add a `Pig`, don't edit `Farm`" actualization shape of `(Actualization p.6–7)` and OCP of `(OOPrinciples p.6–7)`.
- **Clean Architecture layering in JHotDraw.** The drawing model (`Drawing`, `Figure`, `Handle`) plays the **Entities/Use-Cases** role (application-independent + application-specific business rules), while `DrawingView`, Swing components, and tools play the **Interface Adapters / Frameworks & Drivers** role. The Dependency Rule predicts that a "swap the GUI toolkit" change request should ripple only through the outer layers — the same argument the deck makes for the database being a plug-in `(Clean Architecture p.16–18)`.
- **Incorporation & propagation in JHotDraw.** A change request such as "add a new tool" or "add a cashier-login-style new responsibility" `(Actualization p.12–14)` is incorporated as a new class plugged into the existing framework; you then walk the dependency graph (`Tool` → `DrawingEditor` → `DrawingView` …) to propagate secondary changes, ending when consistency is restored `(Actualization p.16–21)`.
- **Impact analysis accuracy.** When you actualize a JHotDraw change, the set of `Figure` / `Tool` / `View` classes you predicted in Impact Analysis can be scored with precision/recall after propagation — the Ericsson method `(Actualization p.24–27)` applied to the case study.

When the exam asks for a JHotDraw example, choose the `Figure` polymorphism hierarchy for OCP/LSP/Polymorphism, an `interface`-segregated handle/tool API for ISP, dependency-injected/`Factory`-created figures for DIP/Creator, and the model-vs-view split for Clean Architecture layering.

Per-principle CASE-study example bank (the lab's portfolio deliverable, pre-assembled)

The lab's portfolio items are "Provide examples of the SOLID principles in context of the CASE study" and "Explain Clean Architecture in context of the CASE Study" `[ActLab]`. A complete portfolio/exam answer for each item follows one fixed pattern: **(1)** state the principle with its slide-exact wording and citation, **(2)** point at a concrete CASE-study (JHotDraw-style) element, **(3)** say *why* this limits actualization cost (incorporation effort and/or propagation reach). Bank of ready answers:

- **SRP** — definition: "A class should have only one reason to change" `(OOPrinciples p.4)`. CASE element: a drawing tool class that only handles its own tool gesture (selection vs. creation vs. text editing live in separate tool classes), the way `UserService` / `SecurityService` split password changing from access checking `(OOPrinciples p.5)`. Why it helps: a change request about one behaviour names one class — Impact Analysis gets easier, the ripple starts smaller.
- **OCP** — definition: "Classes should be open for extension but closed for modification"; couple "between a concrete class and an abstract class or an interface" `(OOPrinciples p.6)`. CASE element: the `Figure` abstraction with shape subtypes; adding a new shape = adding a subclass, exactly the `Validator` interface move that frees `LoanApprovalHandler` from `PersonalLoanValidator` `(OOPrinciples p.7)`. Why: a new-feature change request becomes *pure new code* — the actualization shape with the least propagation `(Actualization p.5–7)`.
- **LSP** — definition: "Subclasses should be substitutable for their base classes" `(OOPrinciples p.8)`. CASE element: every `Figure` subtype must honour the base drawing/bounds contract; a subtype that throws on a base operation would be the `Ostrich.fly()` violation `(OOPrinciples p.9)`. Why: violating subtypes turn polymorphic incorporation into defect injection — clients start special-casing, which *is* propagation.

- **ISP** — definition: "Many specific interfaces are better than a single, general interface"; "Any interface we define should be highly cohesive" `(OOPrinciples p.10)`. CASE element: separate, role-sized interfaces for figures/handles/tools rather than one fat editor interface — the `IUser` / `IRole` / `IUserRole` split `(OOPrinciples p.11)`. Why: a signature change ripples only to clients of that slice.
- **DIP** — definition: "Depend upon abstractions. Do not depend upon concretions"; "DIP tells us how we can adhere to OCP" `(OOPrinciples p.12)`. CASE element: editor classes that receive their collaborators (storage, format, figure factories) through constructor-injected abstractions, as `Payments` receives a `PaymentMethod` `(OOPrinciples p.13)`. Why: swapping a concrete supplier no longer touches the client — that whole category of change requests stops propagating.
- **CRP** — "Favor polymorphic composition of objects over inheritance"; "Delegation can be a better alternative to inheritance" `(OOPrinciples p.14)`. CASE element: a figure that *holds* a reusable behaviour object (e.g., a connection or layout strategy) and delegates, instead of subclassing for every behaviour mix. Why: behaviour can be swapped at run time without fragile deep hierarchies.
- **PLK / Law of Demeter** — operations call only "itself, its parameters, objects it creates, or its contained instance objects" `(OOPrinciples p.15)`. CASE element: no `editor.getView().getDrawing().getFigure(i).getHandle(j)` train-wrecks; the view exposes an operation that does the work. Why: transitive visibility is exactly the invisible coupling that wrecked Ericsson's recall `(Actualization p.27–28)`.
- **Clean Architecture layering** — the CASE study's drawing model (figures, drawing rules) plays Entities/Use-Cases; tool/controller classes are Interface Adapters; the GUI toolkit and file I/O are Frameworks & Drivers `(Clean Architecture p.6–7)`. State the Dependency Rule `(Clean Architecture p.8)` and then make the lab's argument: a "change the GUI" or "change persistence" request is actualized entirely in the outer ring, the same way the database swap stays inside the Entity Gateway Implementation `(Clean Architecture p.16–18)`.
- **GRASP bonus material** (for "which class gets this responsibility?" follow-ups): figure geometry computed by the figure that owns the data (Information Expert `(OOPrinciples p.19)`); a drawing creates its figures because it aggregates them (Creator, with the aggregates/contains tie-break `(OOPrinciples p.20)`); tool/use-case controllers receive UI events, never widget classes themselves (Controller `(OOPrinciples p.23)`); `Figure.draw()` per subtype (Polymorphism `(OOPrinciples p.24)`); a storage adapter between editor and file format (Indirection `(OOPrinciples p.25)`); a `PersistentStorage`-style class for saving drawings (Pure Fabrication `(OOPrinciples p.26)`); a stable interface in front of the anticipated-to-change file format (Protected Variations `(OOPrinciples p.27)`).

Worked Example / Process Walkthrough

Scenario (from the deck): "Add a cashier login to the Point-of-Sale system" — a complete actualization, end to end.

0. Pre-conditions (earlier phases). Initiation produced the change request: *"Create a cashier login that will control the user log in with a username and password."* `(Actualization p.12)`. Concept Location found the launch/authorization entry point; Impact Analysis predicted which model classes (`Store`, `Inventory`, `Item`, `Price`, and the launch code) are in the impact set; Prefactoring cleaned an insertion seam.

1. Decide the technique by size. This is *not* a one-field tweak like the ZIP example `(Actualization p.2–4)`; it adds a brand-new responsibility (authentication). So it is a **larger change** → write a new class separately, then incorporate `(Actualization p.5)`.

2. Implement the new class in isolation. Create a `Cashiers` class holding username/password credentials and a `login(username, password)` operation. Apply SOLID while writing it: - **SRP** — `Cashiers` only authenticates; it does *not* also price items `(OOPrinciples p.4–5)`. - **DIP** — depend on an abstract credential store, injected, not a concrete DB call `(OOPrinciples p.12–13)`; in Clean-Architecture terms the store is reached through an **Entity Gateway/Repository** interface `(Clean Architecture p.7, p.18)`. - **Information Expert / Creator** — the class that *has* the credential data owns `checkAccess`, and whichever class *aggregates* cashiers creates them `(OOPrinciples p.19–20)`.

3. Incorporate (plug it in). Add `Cashiers` to the model and wire the launch path to require `login()` first `(Actualization p.13 → p.14)`. Structurally this is "incorporating a new supplier" — the launch/ `Store` client now points at the new `Cashiers` supplier `(Actualization p.11)`.

4. Propagate the change. Incorporation makes the launch code and any client of "anyone can launch" inconsistent (previously no auth was required `(Actualization p.12)`). Walk the dependency graph — like the `sale` / `register` / `store` propagation `(Actualization p.16–21)` — updating each dependent: the UI must show a login screen (outer **Frameworks & Drivers**/**View** layer only, if the architecture is clean), session handling must check the logged-in cashier, and any code that assumed an always-open till must now branch on auth state. Continue until no inconsistent class remains — **propagation ends** `(Actualization p.21)`.

5. Keep the ripple short with Clean Architecture. Because the login *use case* (interactor) is insulated from the UI and DB `(Clean Architecture p.4, p.8)`, the credential-store choice (SQL vs. NoSQL) and the login-screen look are *details* that plug in `(Clean Architecture p.16–18)`; propagation is bounded to the gateway implementation and the view, never the business entities.

6. Verify the prediction (precision/recall). After propagation, compare the classes you actually touched against the Impact-Analysis prediction. Build the confusion matrix `(Actualization p.24)`, compute **precision = TP/(TP+FP)** and **recall = TP/(TP+FN)** `(Actualization p.26–27)`. If recall is low (you missed dependent classes, the Ericsson 32% trap `(Actualization p.27)`), that is the "moment of truth" telling you Impact Analysis under-estimated `(Actualization p.23, p.28)`.

7. Hand off to Postfactoring. With the feature working, clean up (Postfactoring/Lecture 6), then Conclusion — all under continuous Verification `(Actualization p.1)`.

Second worked example — the deck's own taxCategory incorporation, traced end to end

The deck's *other* complete actualization is the `taxCategory` walk `(Actualization p.16–21)`; tracing it in process terms makes a compact model answer:

1. The change (implied). A tax-category concept must be added to the Point-of-Sale model — a *larger* change, so a new class `taxCategory` is written separately `(Actualization p.5)`.

2. Incorporation. `taxCategory` (blue) is plugged in with `item` as the incorporation point — the grey arrow on the slide runs from `taxCategory` into `item` `(Actualization p.16)`. Structurally this is "incorporating a new supplier" `(Actualization p.11)`: `item` is the first old client of the new class.

3. Propagation, step by step (colour semantics in Concept 20): `item` is modified (red) → its neighbours `store` and `saleLineItem` are marked (green) `(Actualization p.17)`; `store` is inspected and needs nothing (grey) `(Actualization p.18)`; `saleLineItem` is modified (red) → `sale` marked `(Actualization p.19)`; `sale` is modified (red) → `register` marked `(Actualization p.20)`; `register` is inspected and needs nothing (grey) — no marked classes remain, **propagation ends** `(Actualization p.21)`.

4. Score the prediction (practice computation, applying the deck's method to the deck's walk). Suppose Impact Analysis had predicted the impact set {`item`, `saleLineItem`} — a plausible guess that misses the indirect dependent `sale`. Actual changed set (the red classes): {`item`, `saleLineItem`, `sale`}. Then TP = 2, FP = 0, FN = 1, and with the remaining old classes (`store`, `register`) as true negatives, TN = 2. Precision = TP/(TP+FP) = 2/2 = **100%**; Recall = TP/(TP+FN) = 2/3 ≈ **67%** `(Actualization p.26–27)`. Same signature as Ericsson — perfect precision, imperfect recall — because the easiest classes to miss are the ones reached only *transitively* through the dependency graph `(Actualization p.28)`. (The numbers here are a constructed drill; the Ericsson 30/0/42/64 figures are the deck's real data `(Actualization p.24–25)`.)

5. The design moral. `sale` changed only because `saleLineItem` changed, which changed only because `item` changed. Every seam that decouples a link in that chain — an interface in front of `item` (Protected Variations `(OOPrinciples p.27)`), telling-not-asking between `sale` and `saleLineItem` (PLK `(OOPrinciples p.15)`) — would have stopped the ripple a step earlier. That is the lab's whole thesis in one sentence: clean principles shorten exactly this walk `[ActLab]`.

Definitions & Terminology

Term	Definition	Source
Actualization	*What:* the change-process phase that implements new functionality + incorporates it into old code + propagates secondary changes. *Used for:* turning a change request into working, integrated code — the only phase that writes production source.	`(Actualization p.1)`; `[ActLab]`
Small change	*What:* a change done directly inside existing class code (e.g. widen `zip[5]` → `zip[9]`). *Used for:* localized edits that need no new structure; minimal/no incorporation.	`(Actualization p.2–4)`
Larger change	*What:* new classes written separately, then plugged in; may ripple. *Used for:* features that add new structure — built and tested in isolation first, so they are correct before wiring in.	`(Actualization p.5)`
Incorporation	*What:* plugging newly written classes into the existing code as components. *Used for:* connecting isolated new code to the live system; its *shape* (local/composite/supplier) sets how far change then ripples.	`(Actualization p.5, p.8)`
Ripple effect	*What:* a change spreading to other components via dependencies. *Used for:* (a thing to *limit*) — naming why one edit forces many; low coupling/clean architecture exist to shrink it.	`(Actualization p.5)`
Change propagation	*What:* seeking out and updating every place in old code needing secondary modification, until consistency is restored. *Used for:* making an incorporated change actually correct (not just compiling); also applies to deletions.	`(Actualization p.16–22)`; `[ActLab]`
Moment of truth	*What:* propagation confirms/refutes Impact-Analysis predictions. *Used for:* measuring how good the earlier impact estimate was — the basis of precision/recall scoring.	`(Actualization p.23)`
Precision	*What:* TP/(TP+FP); fraction of predicted-changed classes that really changed (Ericsson = 100%). *Used for:* measuring *false-alarm* rate — high precision means nothing was flagged needlessly.	`(Actualization p.26)`
Recall	*What:* TP/(TP+FN); fraction of actually-changed classes that were predicted (Ericsson = 32%). *Used for:* measuring *misses* — the hard metric, since low recall (forgotten classes) is what blows schedules.	`(Actualization p.27)`
Under-estimation	*What:* systemic; a consequence of (dependency) invisibility; makes planning hard. *Used for:* explaining *why* recall is low — you can't predict dependencies you can't see.	`(Actualization p.28)`

Term	Definition	Source
Software architecture	*What:* high-level structures of a system + the discipline of creating them (Hexagonal, Onion, Clean). *Used for:* organising the *whole* system and governing inter-part dependencies (vs. a pattern's local scope).	`(Clean Architecture p.2)`
Design pattern	*What:* re-usable form of a solution to a design problem (Singleton, Observer, MVC/MVVM/MVP). *Used for:* solving one recurring *local* collaboration problem among a few classes.	`(Clean Architecture p.2)`; `[GHJV94]`
Clean Architecture	*What:* Martin's concentric-layer architecture insulating business rules from technical details. *Used for:* making whole categories of change (swap UI/DB/framework) cheap by keeping volatile details outside stable rules.	`(Clean Architecture p.5)`; `[Martin]`
Dependency Rule	*What:* source-code dependencies point only inward (toward stable abstractions). *Used for:* guaranteeing the four independences — details depend on abstractions, never the reverse (DIP at architecture scale).	`(Clean Architecture p.8)`
Entities	*What:* innermost layer; enterprise-wide, application-independent business rules; least likely to change. *Used for:* holding the most-stable domain knowledge as the bedrock all other layers depend on.	`(Clean Architecture p.7, p.9)`
Use Cases / Interactors	*What:* application-specific business rules; isolated from DB, frameworks, UI. *Used for:* orchestrating the entities through one application workflow ("the dance of the entities") free of delivery tech.	`(Clean Architecture p.7, p.9)`
Interface Adapters	*What:* convert data between inner format and DB/web; includes Presenters, ViewModels, Gateways/Repositories. *Used for:* translating so inner layers speak domain objects and outer layers speak JSON/SQL/pixels.	`(Clean Architecture p.7)`
Frameworks & Drivers	*What:* outermost layer: web framework, DB, UI, HTTP client. *Used for:* corralling all volatile, replaceable, vendor-supplied technology at the edge as plug-in "details."	`(Clean Architecture p.7)`
Boundary `<I>`	*What:* interface/protocol through which data enters/leaves the interactor. *Used for:* inverting the dependency so the UI depends inward on an abstraction the interactor owns.	`(Clean Architecture p.9)`
Request / Response Model	*What:* plain data structures (primitives) carrying input into / results out of the interactor. *Used for:* moving data across boundaries without framework types, keeping inner layers ignorant of the outside.	`(Clean Architecture p.10, p.12, p.15)`
Presenter / View Model / View	*What:* Presenter translates Response Model → View Model (strings/flags); View just displays it ("so stupid you don't test it"). *Used for:* pushing all formatting out of entities/View into one testable place, leaving the View logic-free.	`(Clean Architecture p.15)`
Entity Gateway / Repository	*What:* interface (declared with business rules) implemented in the detail layer to reach the Database API. *Used for:* letting use cases depend inward on an abstraction so the DB can be swapped without touching them (DIP/Protected Variations).	`(Clean Architecture p.7, p.18)`

Term	Definition	Source
"Database is a detail"	*What:* DB/UI/frameworks are plug-ins to business rules, not the center. *Used for:* bounding the ripple — "swap SQL→NoSQL" is actualized only in the gateway impl, never the entities.	`(Clean Architecture p.16–18)`
SOLID	*What:* five OO design principles; coined by Martin (2000 paper), acronym by Michael Feathers. *Used for:* structuring classes/dependencies for low coupling and change-by-extension — i.e. maintainability.	`(OOPrinciples p.2–3)`; `[Martin]`
SRP — Single Responsibility	*What:* a class should have only one reason to change / a single responsibility. *Used for:* shrinking change propagation — one responsibility per class means a request touches few classes. *Fix:* extract the extra responsibility into its own class.	`(OOPrinciples p.4)`
OCP — Open/Closed	*What:* open for extension, closed for modification; couple at the abstract level. *Used for:* adding features without re-editing tested code, confining ripple to the new class. *Fix:* depend on an abstraction; add variants as new implementations.	`(OOPrinciples p.6)`
LSP — Liskov Substitution	*What:* subclasses must be substitutable for their base classes (a *behavioural* contract, not just compilation). *Used for:* making polymorphism/OCP safe — a misbehaving subtype injects defects instead of absorbing change. *Fix:* restructure so unsupported capabilities live only on the subtypes that have them.	`(OOPrinciples p.8)`
ISP — Interface Segregation	*What:* many specific, highly-cohesive interfaces beat one general interface; no client depends on methods it doesn't use. *Used for:* confining a signature change to the clients of that slice. *Fix:* split a fat interface along client-role lines.	`(OOPrinciples p.10)`
DIP — Dependency Inversion	*What:* depend upon abstractions, not concretions; tells us how to adhere to OCP. *Used for:* insulating a class from concrete-collaborator choices/changes; it *is* the Dependency Rule at class scale. *Fix:* depend on an interface, inject the concrete (constructor injection).	`(OOPrinciples p.12)`
CRP — Composite Reuse	*What:* favor (polymorphic) composition/delegation over inheritance as the *primary* reuse mechanism. *Used for:* localising change (swap a delegate at run time) and avoiding fragile-base-class breakage in deep hierarchies. *Fix:* extract reused behaviour into a held collaborator and delegate. (Not "never inherit" — true is-a still uses inheritance.)	`(OOPrinciples p.14)`
PLK / Law of Demeter	*What:* an operation calls only methods of itself, its parameters, objects it creates, or its instance fields; avoid transitive visibility. *Used for:* suppressing the invisible transitive coupling that defeats Impact Analysis. *Fix:* replace `a.getB().getC().do()` train-wrecks with *tell-don't-ask* (`a.do()`).	`(OOPrinciples p.15)`
GRASP	*What:* General Responsibility Assignment Software Patterns; 9 patterns by Craig Larman. *Used for:* deciding *which class* should hold each responsibility, so the result is low-coupled and cohesive.	`(OOPrinciples p.16)`; `[Larman04]`
Information Expert	*What:* assign a responsibility to the class that has the information to fulfill it. *Used for:* keeping behaviour next to its data → low coupling, high cohesion (the default GRASP). *When:* "who already knows the data?" → put the method there.	`(OOPrinciples p.19)`
Creator	*What:* class B creates A if B aggregates/contains/records/closely-uses A or has A's init data. *Used for:* deciding who instantiates whom so creation follows existing structure. *When:* prefer the class that *contains/aggregates* A as its creator.	`(OOPrinciples p.20)`
Low Coupling	*What:* an *evaluative* pattern — assign responsibilities to keep coupling low (high coupling = fragile, hard to reuse/understand). *Used for:* directly minimising change propagation. *When:* a tie-breaker — pick the assignment adding the fewest dependencies.	`(OOPrinciples p.21)`

Term	Definition	Source
High Cohesion	*What:* an *evaluative* pattern — assign responsibilities to keep cohesion high (low cohesion = hard to comprehend/reuse/maintain, "delicate"). *Used for:* keeping each class single-purpose (class-level SRP). *When:* if an assignment would make a class do unrelated work, place it elsewhere.	`(OOPrinciples p.22)`
Controller	*What:* a non-UI class (façade or use-case/session) handles system events; UI/view classes delegate to it. *Used for:* a single UI-to-domain entry point so business logic doesn't leak into views. *Trap:* window/widget/view/document classes must *not* be the controller.	`(OOPrinciples p.23)`
Polymorphism (GRASP)	*What:* when behavior varies by type, use polymorphic operations defined on a base class/interface. *Used for:* replacing type-switch conditionals so new variants are new types, not new branches (= OCP). *When:* you see `if type==…`.	`(OOPrinciples p.24)`
Indirection	*What:* use an intermediate object to decouple components; beware transitive visibility. *Used for:* keeping two collaborators independently changeable via a mediator/adapter (operationalises PLK). *When:* two parts must interact but should not know each other.	`(OOPrinciples p.25)`
Pure Fabrication	*What:* a made-up, highly-cohesive class (not a domain concept) to gain cohesion/low coupling/reuse (e.g. `PersistentStorage`). *Used for:* housing non-domain concerns (persistence, logging) without polluting domain entities. *When:* Expert would point at a domain class but doing so would wreck its cohesion → fabricate. (= Clean Arch Gateway.)	`(OOPrinciples p.26)`
Protected Variations	*What:* wrap predicted points of variation behind a stable interface (access view, broad sense). *Used for:* the unifying pattern — stops change rippling past the seam. *When:* anticipate what will change (DB vendor, UI toolkit, a rule) and front it with a stable interface. (OCP/DIP/Indirection are special cases.)	`(OOPrinciples p.27)`

Fine-print terms and diagram vocabulary

Term	Definition	Source
Composite responsibility	The responsibility of a class (e.g. `Farm`) that is realised through a *family* of polymorphic parts; "the composite responsibility of Farm was extended by the concept Pig" — extended by adding a subtype, not by editing `Farm`.	`(Actualization p.7)`
Incorporation point	The old-code class first modified to connect the new class (e.g. `item` for `taxCategory`; `Store` for `Cashiers`) — where the propagation walk starts.	`(Actualization p.16, p.14)`
Propagation colours (deck convention)	Blue = newly incorporated class; green = marked for inspection; red = modified (secondary change confirmed); grey = inspected, no change needed. Walk ends when no green remains.	`(Actualization p.16–21)`
Visited vs. changed	A class can be inspected during propagation and need no edit (`store`, `register` end grey) — only the red (modified) classes count as "actually changed" in the precision/recall tally.	`(Actualization p.17–21, p.24)`
Façade controller	GRASP Controller variant: a class representing "the overall system, device, or subsystem" that receives system events.	`(OOPrinciples p.23)`
Use-case / session controller	GRASP Controller variant: a class representing "a use case scenario within which the system event occurs"; the *same* controller class handles all system events of one scenario; "a session is an instance of a conversation with an actor."	`(OOPrinciples p.23)`

Term	Definition	Source
Creator tie-break	"If more than one option applies, prefer a class B which aggregates or contains class A" — containment beats records/closely-uses/has-init-data.	`(OOPrinciples p.20)`
Input vs. output boundary	The *input* boundary is implemented by the interactor itself (so the controller can call inward); *output* boundaries are "implemented by other objects" (the presenter) so results can flow outward without an outward dependency.	`(Clean Architecture p.9, p.14)`
"Dance of the entities"	The deck's phrase for the interactor's job: it "controls the dance of the entities (ex: creates the order, modify the customer, etc)" — first forward (executing), then "in the opposite direction" (gathering results).	`(Clean Architecture p.11–12)`
Stability pyramid ("Level")	Slide-8 triangle: Entities at apex → Use Cases → Controllers/Gateways/Presenters → Devices/Web/DB/External Interfaces/UI at base; axis runs "Abstract, General, Rarely Change" → "Concrete, Specific, Change Frequently."	`(Clean Architecture p.8)`
Data flow vs. dependency rule	Two distinct arrows on slide 8: dashed "Data Flow" runs *through* the layers (UI → Presenter → Use case → Entity → Repository → Data source); solid "Dependency rule" arrows point only *inward*.	`(Clean Architecture p.8)`
Figures 1.1–1.8	The deck's numbered diagrams: 1.1 static cast (p.9); 1.2 request model created, steps 1–3 (p.10); 1.3 dance of entities, step 4 (p.11); 1.4 response model gathered, step 5 (p.12); 1.5 response delivered, step 6 (p.13); 1.6 Controller/Presenter/View Model/View assembly (p.14); 1.7 Entity Gateway (p.18); 1.8 full assembly (p.19).	`(Clean Architecture p.9–19)`
"Test it with your eyes"	The View is so logic-free that visual inspection replaces unit testing — the deck's literal parenthetical.	`(Clean Architecture p.15)`
Transitive visibility	Obtaining a reference "by calling a method on another object" and then calling methods on it — what PLK forbids and what Indirection warns to "beware of."	`(OOPrinciples p.15, p.25)`

Common Pitfalls / Gotchas

1. **Confusing precision and recall.** Precision = TP/(TP+FP) (no false alarms); recall = TP/(TP+FN) (no misses). Ericsson had **100% precision but 32% recall** `(Actualization p.26–27)` — flag the *denominator*: precision divides by *predicted-changed*, recall by *actually-changed*. False **negatives** (predicted-unchanged-but-changed) are the dangerous ones and they live in the recall denominator.
2. **Reading the confusion matrix the wrong way.** In the Ericsson table, **rows = Predicted, columns = Actual**; FN = 64 sits in *Predicted-Unchanged / Actual-Changed* `(Actualization p.24–25)`. Total = 136. Don't swap FP and FN.
3. **Thinking small changes have no propagation.** Even a direct edit can ripple; and **deletion** definitely propagates — "all references to the deleted functionality must be deleted" `(Actualization p.22)`.
4. **Treating "architecture" and "design pattern" as synonyms.** The deck explicitly separates them: MVC/MVVM/MVP/Singleton/Observer are *patterns*; Hexagonal/Onion/Clean are *architectures* `(Clean Architecture p.2)`.

5. **Getting the Dependency Rule backwards.** Dependencies point **inward** toward stable abstractions (Entities), never outward `(Clean Architecture p.8)`. Outer layers know inner layers; inner layers must know *nothing* about outer layers.
6. **Putting business rules in the database / UI.** "Business rules shouldn't be written in stored procedures" and the DB is "a detail / plug-in" `(Clean Architecture p.17)`. Likewise the View should be "so stupid you don't test it" `(Clean Architecture p.15)`.
7. **DIP ≠ DI framework.** Dependency *Inversion* is the principle (depend on abstractions); the deck's example achieves it with plain **constructor injection** `(OOPrinciples p.13)` — no framework needed. Also remember the slide's key line: **DIP is how you adhere to OCP** `(OOPrinciples p.12)`.
8. **LSP is about behavioural contract, not just compiling.** `Ostrich extends Bird` *compiles* but throws on `fly()` — it violates LSP because it breaks substitutability `(OOPrinciples p.9)`.
9. **ISP vs SRP confusion.** SRP is about a *class* having one reason to change `(OOPrinciples p.4)`; ISP is about *interfaces* being client-specific and cohesive `(OOPrinciples p.10)`. They rhyme but are different scopes.
10. **GRASP Controller excludes UI classes.** "window, applet, widget, view, document" classes must **not** handle system events — they delegate to a controller `(OOPrinciples p.23)`. A common exam trap.
11. **CRP overstated.** The slide says favor composition and calls inheritance-as-primary-reuse "catastrophic" `(OOPrinciples p.14)`, but inheritance is still correct for genuine is-a polymorphism (the `Farm` / `Pig` and OCP examples *use* inheritance). Don't claim "never inherit."
12. **GRASP count.** There are exactly **nine** GRASP patterns `(OOPrinciples p.18)`; SOLID is **five** `(OOPrinciples p.3)`. Don't merge the lists.
13. **Mis-attributing authorship.** SOLID = Martin's 2000 paper, acronym = Michael Feathers `(OOPrinciples p.2)`; GRASP = Craig Larman, *Applying UML and Patterns* `(OOPrinciples p.17)`; Clean Architecture = Robert C. Martin `(Clean Architecture p.5)`.

More traps from the fine print (14–25)

14. **Confusing data flow with dependency direction.** On `(Clean Architecture p.8)` the dashed "Data Flow" arrow runs straight *through* every layer (UI → Presenter → Use case → Entity → Repository → Data source) while the solid "Dependency rule" arrows point only inward. Data crosses boundaries in both directions; source-code dependencies never point outward. "Dependencies follow the data" is the classic wrong answer.
15. **Reversing boundary ownership.** The **input** boundary is implemented *by the interactor* ("Data comes into interactors through boundary … which is implemented by the interactor"); **output** boundaries are "implemented by other objects" — the presenter side `(Clean Architecture p.9, p.14)`. Swapping these breaks the whole dependency-inversion story.
16. **Passing entities across the boundary.** The Request Model "contains only primitives types" `(Clean Architecture p.10)` and the Response Model's data "is not presentable" but is still plain data (date object, money object) `(Clean Architecture p.15)`. Neither is an Entity, and neither contains framework types — that is what keeps inner layers ignorant of the outside.
17. **Forgetting the pattern provenance labels.** The Interface Adapters layer "includes Presenters from **MVP**, ViewModel from **MVVM**, and Gateways (also known as **Repositories**)" `(Clean Architecture p.7)` — three names, three origins; exam answers that say "Presenter from MVVM" lose the mark.
18. **Treating the Gateway as "the database layer."** The Entity Gateway `<I>` is declared *with* the business rules (above the boundary line in Figure 1.7); only the *Implementation* lives below the line next to the Database API `(Clean Architecture p.18)`. The interface's location is the entire point.
19. **Counting visited classes as changed.** In the propagation walk, `store` and `register` are inspected but end **grey** (no modification) `(Actualization p.18, p.21)`. Only red (modified) classes belong to the

"actual changed" set when scoring impact analysis `(Actualization p.24)`. Visited ≠ changed.

20. **Confusing the new class with the impacted old classes.** `taxCategory` (blue) and `Cashiers` (dark grey) are *new code* `(Actualization p.16, p.13)`; the ripple effect is about secondary modifications in the *old* components ("The change can propagate to other components of the system" `(Actualization p.5)`). Don't list the new class as a "propagated" change.
21. **Dropping Creator's tie-break.** When several classes qualify as creator, "prefer a class B which aggregates or contains class A" `(OOPrinciples p.20)` — aggregation/containment outrank "records," "closely uses," and "has the initializing data."
22. **Merging ISP and DIP because both "reduce coupling."** ISP *splits interfaces* along client roles ("many specific interfaces are better than a single, general interface" `(OOPrinciples p.10)`); DIP *redirects dependencies* toward abstractions ("couple at the abstract level" `(OOPrinciples p.12)`). One reshapes contracts, the other re-aims edges.
23. **Quoting CRP as just "composition over inheritance."** The slide's wording is "Favor **polymorphic composition** of objects over inheritance" `(OOPrinciples p.14)` — the delegate is reached through an abstraction (hence *polymorphic*), which is what makes it swappable.
24. **Citing the wrong Martin source for SOLID.** The theory comes from his **2000 paper "Design Principles and Design Patterns"** `(OOPrinciples p.2)`, not from the *Clean Code* book; the deck's "(Clean Code)" tag on `(Clean Architecture p.5)` identifies the man, not the SOLID source.
25. **Misremembering the ZIP example.** Only `zip[5]` widens to `zip[9]`; `state[2]`, `name`, `streetAddress`, `city`, and `move()` are unchanged `(Actualization p.2–3)` — and the deck shows the after-state twice (p.3 and p.4 are the same class) to underline that a small change *stays* local.

Exam Focus

Most likely to be tested: - **Define Actualization** and list its three sub-activities (implement, incorporate, propagate) `[ActLab]` `(Actualization p.1)`, and place it in the seven-phase process. - **Compute precision and recall** from a given confusion matrix and interpret the result (the Ericsson numbers are prime exam fodder: 100% / 32%, total 136) `(Actualization p.24–27)`. Be able to explain *why* recall is low (invisible dependencies → under-estimation) `(Actualization p.28)`. - **Explain change propagation and when it ends** (system consistent again), and that deletion propagates too `(Actualization p.16–22)`. State that propagation is the "moment of truth" for Impact Analysis `(Actualization p.23)`. - **State and define all five SOLID principles**, each with its one-line definition and a before/after example `(OOPrinciples p.3–13)`. Know that **DIP enables OCP** and that **LSP = substitutability**. - **Define CRP and PLK / Law of Demeter** `(OOPrinciples p.14–15)`. - **Name and define the nine GRASP patterns** `(OOPrinciples p.18–27)`; be ready to pick the right one for a scenario (who creates X? → Creator; who handles a system event? → Controller; behavior varies by type? → Polymorphism; wrap an unstable point? → Protected Variations). - **Draw the Clean Architecture circles**, label the four layers, and state the **Dependency Rule** `(Clean Architecture p.5–8)`. List the four characteristics (testable; independent of UI/DB/frameworks) `(Clean Architecture p.3–4)`. - **Explain "the database is a detail"** and the Entity Gateway/Repository, and distinguish architecture from design pattern `(Clean Architecture p.2, p.16–18)`.

High-value synthesis questions (and the answer spine): - *"How do SOLID / Clean Architecture make actualization cheaper?"* → They lower coupling and put variation behind stable interfaces (OCP/DIP/Protected Variations, Dependency Rule), so incorporating new code ripples less; change propagation terminates sooner and Impact-Analysis recall improves. - *"Give a JHotDraw/CASE example of each SOLID principle and of Clean Architecture layering"* — this is the explicit **lab/portfolio deliverable** `[ActLab]`; prepare the `Figure` hierarchy (OCP/LSP/Polymorphism), segregated tool/handle interfaces (ISP),

injected/factory-created figures (DIP/Creator), single-responsibility tools (SRP), and the model-vs-Swing-view split as the Entities/Use-Cases vs. Frameworks-&-Drivers layering with the Dependency Rule. - *"Why is impact routinely under-estimated?"* → invisibility of dependencies; makes planning difficult; recall (not precision) is the hard metric `(Actualization p.27–28)`.

Memory hooks: SOLID = **S**ingle / **O**pen-closed / **L**iskov / **I**nterface-seg / **D**ependency-inversion. GRASP nine = "**I** **C**an **L**ift **H**eavy **C**rates, **P**lease **I**nspect **P**roperly, **P**al" → **I**nfo Expert, **C**reator, **L**ow coupling, **H**igh cohesion, **C**ontroller, **P**olymorphism, **I**ndirection, **P**ure fabrication, **P**rotected variations. Clean layers (out→in): **F**rameworks → **I**nterface adapters → **U**se cases → **E**ntities ("**FIUE**", dependencies flow the *other* way, inward).

Numeric drills — precision/recall beyond the Ericsson numbers

The method `(Actualization p.24–27)` must work for *any* matrix the examiner invents. Drill the mechanics (all derived practice applying the deck's formulas):

1. **The real data** `(Actualization p.24–25)`: TP = 30, FP = 0, TN = 42, FN = 64; total 136. Precision = 30/30 = **100%**; Recall = 30/94 ≈ **32%** `(Actualization p.26–27)`.
2. **What if there had been false alarms?** Keep everything else, set FP = 10: precision = 30/(30+10) = **75%**; recall unchanged at 32% — precision and recall move independently because they share only TP.
3. **What if recall had been better?** Set FN = 32 (half as many misses): recall = 30/(30+32) ≈ **48%**; precision unchanged at 100%.
4. **From the propagation walk** (Concept 20 tally): if the prediction was {`item`, `saleLineItem`} and the red set is {`item`, `saleLineItem`, `sale`}, then TP = 2, FP = 0, FN = 1 → precision **100%**, recall ≈ **67%**.
5. **Reading-direction check**: the Ericsson matrix has **rows = Predicted, columns = Actual** `(Actualization p.24)`; FN (64) sits at row *Predicted-Unchanged*, column *Actual-Changed*. If an exam table is transposed, recompute — never pattern-match cell positions.
6. **Interpretation sentence to memorise**: "Programmers estimated that the changes will impact only about a third of all classes that actually changed — missed the other two thirds!" `(Actualization p.27)`; the cause is invisibility of dependencies, and it "makes planning difficult" `(Actualization p.28)`.

Answer skeletons for the drawing/essay questions

- **"Draw Clean Architecture."** Four concentric rings, inner→outer: Entities (Enterprise Business Rules), Use Cases (Application Business Rules), Interface Adapters (Controllers, Presenters, Gateways), Frameworks & Drivers (Web, DB, Devices, UI, External Interfaces) `(Clean Architecture p.6–7)`. Add inward arrows + label "Dependency Rule" `(Clean Architecture p.8)`. Close with the four characteristics: testable, independent of UI, of database, of frameworks/external entities `(Clean Architecture p.3)`.
- **"Explain a use-case execution."** Recite steps 1–6 verbatim-ish from Figures 1.2–1.5: button click → Request Model (primitives only) → input boundary (implemented by interactor) → dance of the entities → Response Model → output boundary → Presenter → View Model → View `(Clean Architecture p.10–15)`.
- **"Explain actualization."** Definition (implement + incorporate + propagate) `[ActLab]` `(Actualization p.1)`; size decides technique (ZIP edit vs. new class) `(Actualization p.2–5)`; three incorporation shapes `(Actualization p.9–11)`; propagation walk + termination `(Actualization p.16–21)`; deletion propagates `(Actualization p.22)`; moment of truth + precision/recall `(Actualization p.23–27)`.
- **"How do the principles help maintenance?"** Pick three seams and name the principle that builds each: an abstraction in front of variants (OCP/DIP `(OOPrinciples p.6, p.12)`), responsibilities next to

their data (Information Expert `(OOPrinciples p.19)`), a stable interface around predicted change (Protected Variations `(OOPrinciples p.27)`) — each seam is a place the propagation walk stops early.

Slide-by-Slide Source Walkthrough

This section accounts for **every page of every source PDF** in order, so any slide an exam question references can be located instantly. (Diagram descriptions are from the page images; quoted text is verbatim from the slides.)

Actualization.pdf — all 28 slides

- **p.1 — "8 Actualization."** Definition bullets: "Programmers implement the new functionality — according to change request"; "The process of actualization varies — depends on the size of the change." Right margin: the phase ladder Initiation → Concept Location → Impact Analysis → Prefactoring → **Actualization** → Postfactoring → Conclusion with the vertical **V-E-R-I-F-I-C-A-T-I-O-N** bar spanning all phases. Footer (as on every slide): "Software Engineering: The Current Practice Ch. 8".
- **p.2 — "Small changes."** "Done directly in old code." Code: `class Address { public move(); protected String name; protected String streetAddress; protected String city; protected char state[2], zip[5]; };` — the before-state with a 5-character ZIP.
- **p.3 — "Small changes."** Identical class but `zip[9]` — the after-state: one field widened in place; no new structure.
- **p.4 — "Small changes."** Repeats the p.3 after-state (`zip[9]`) unchanged — the deck dwells on the result to stress that nothing else moved.
- **p.5 — "Larger changes."** "Programmers implement the new classes separately from the old code"; "The new code is plugged into the existing code — incorporation"; "The change can propagate to other components of the system — ripple effect." (The slide's original has a doubled word, "into the the existing code.")
- **p.6 — "Polymorphism."** Old Code / New Code split frame: `Farm`, `Cow`, `Sheep` and base `FarmAnimal` in Old Code; grey `Pig` in New Code with a generalization (hollow-triangle) edge to `FarmAnimal`. `Farm` associates with `FarmAnimal`.
- **p.7 — "Polymorphic class."** Code: `class Pig : public FarmAnimal { public: void makeSound() {cout<<"Oink";} };` plus "Farm now can declare objects of the type Cow, Sheep, or Pig — the composite responsibility of Farm was extended by the concept Pig."
- **p.8 — "Adding New Component."** "Implement the new classes separately from the clients in the old code — new classes assume the responsibilities demanded by the change request"; "New classes are plugged as components into the appropriate place of the existing code — incorporation"; "Change propagation."
- **p.9 — "New responsibility is local."** Two-box frame; old code = five classes in a two-level composition hierarchy (filled diamonds); the grey new class stands alone with **no edges** — fully self-contained.
- **p.10 — "New responsibility is composite."** Same frame; the grey new class now holds **two composition edges into the two bottom old classes** — a composite whose parts are existing old code.
- **p.11 — "Incorporating new supplier."** Adds the reverse wiring: the **top old class aggregates the new class** (composition diamond crossing the boundary), while the new class still aggregates the two bottom old classes.
- **p.12 — "Point of Sale."** "The old application did not require authorization — anyone was able to launch it"; change request: *"Create a cashier login that will control the user log in with a username and*

password."

- **p.13 — "PoS + new class."** `Cashiers` (dark grey) standalone; model chain `Store` ◆— `Inventory` ◆— `Item` ◆— `Price`.
- **p.14 — "Incorporation of Cashier."** New association `Store` ◆— `Cashiers`; `Store` shaded light grey as the modified incorporation point; `Inventory` / `Item` / `Price` untouched.
- **p.15 — "Replacement of a class."** Same five-class frame as p.11, but one old middle class is crossed out (X); the new grey class takes over its role and connections.
- **p.16 — "Example incorporation."** Class graph `register` – `store` – `item` (top), `sale` – `saleLineItem` (bottom), edges `register` – `sale`, `item` – `saleLineItem`; new blue `taxCategory` linked to `item`; grey arrow `taxCategory` → `item`; `item` green. Footer credit "© 2012 Václav Rajlich".
- **p.17 — "Change propagation."** `item` red; black arrows from `item` into `store` (green) and `saleLineItem` (green).
- **p.18 — "Change propagation."** `store` grey (inspected, unchanged); `saleLineItem` still green.
- **p.19 — "Change propagation."** `saleLineItem` red; arrow into `sale` (green).
- **p.20 — "Change propagation."** `sale` red; arrow up into `register` (green).
- **p.21 — "Change propagation ends."** `register` grey; final state: red = {`item`, `saleLineItem`, `sale`}, grey = {`store`, `register`}, blue = `taxCategory`.
- **p.22 — "Deletion of obsolete functionality."** "Also causes change propagation"; "All references to the deleted functionality must be deleted — secondary changes propagate to other classes."
- **p.23 — "Underestimated Impact Set."** "Impact analysis estimates which classes are impacted"; "Change propagation modifies the code of impacted classes — change propagation is the moment of truth — it confirms or refutes the predictions of impact analysis — accuracy of impact analysis predictions is important for software managers."
- **p.24 — "Ericsson Radio Systems."** The 2×2 matrix (rows Predicted, columns Actual): Predicted-Unchanged/Actual-Unchanged 42, Predicted-Unchanged/Actual-Changed 64, Predicted-Changed/Actual-Unchanged 0, Predicted-Changed/Actual-Changed 30; "total number of classes = 42 + 0 + 64 + 30 = 136."
- **p.25 — "Categories."** "true positives = 30; false positives = 0; true negatives = 42; false negatives = 64."
- **p.26 — "Precision."** "Used in the information retrieval"; "Precision = (true positives)/(true positives + false positives)"; "Ericson, precision = 30/(30 + 0) = 1 = 100%." (The slide spells the company "Ericson" here.)
- **p.27 — "Recall."** "Recall = (true positives)/(true positives + false negatives)"; "Ericson recall = 30/(30 + 64) = 0.32 = 32%"; "Programmers estimated that the changes will impact only about a third of all classes that actually changed — missed the other two thirds!"
- **p.28 — "Underestimation."** "Common in software engineering — consequence of invisibility"; "Makes planning difficult"; "Common in other field also."

Clean Architecture.pdf — all 20 slides

- **p.1 — Title.** "Clean Architecture — Jan Sørensen."
- **p.2 — "Design Patter VS Architecture"** (sic — the slide really says "Patter"). Two definitions: "**Design pattern**: A design pattern is the re-usable form of a solution to a design problem. (Singleton - Observer - MVC - MVVM - MVP)"; "**Software architecture**: Software architecture refers to the high level structures of a software system and the discipline of creating such structures and systems. (Hexagonal architecture - Onion architecture - The Clean architecture)."
- **p.3 — "Succesful SW Architecture"** (sic — single "s" in "Succesful"). Four bullets: "Testable; Independent of the UI; Independent of the Database; Independent of Frameworks and External Entities."

- **p.4 — "Characteristics."** Five expanded items (right edge clipped on the slide; clipped words completed from canonical Martin wording): "Independent of frameworks. Libraries and frameworks should [be] use[d] as tools, rather than havi[ng to cram your system into their constraints]. / Testable. Business rules have tests that are independent of UI, Database[, and any external element]. / Independent of UI. UI can change easily, without changing other system compone[nts]. / Independent of Database. The database can be switched from RDBS to NoSQL or any oth[er database]. / Independent of any external agency. Business rules should know nothing about the outside world."
- **p.5 — "The Clean Architecture."** Photo of Robert C. Martin captioned "Robert C. Martin (Clean Code)", beside a small version of the concentric-circle diagram with its four-colour legend.
- **p.6 — "The Clean Architecture."** The circle diagram large: outer blue ring labelled Devices, Web, DB, UI, External Interfaces; green ring labelled Controllers, Gateways, Presenters; red ring Use Cases; yellow core Entities. Legend: Enterprise Business Rules (yellow), Application Business Rules (red), Interface Adapters (green), Frameworks & Drivers (blue).
- **p.7 — "The Clean Architecture."** The four layer definitions in prose: "**Entities** are *enterprise-wide* business rules that encapsulate the most general business rules, these rules are the least likely to change. / **Use cases** are also called interactors and stand for *application-specific* business rules of the software. This layer is isolated from changes to the database, common frameworks, and the UI. / **Interface adapters** convert data from a convenient format for entities and use cases to a format applicable to databases and the web, for example. This layer includes Presenters from MVP, ViewModel from MVVM, and Gateways (also known as Repositories). / **Frameworks and drivers** are the outermost layer, which consists of the web framework, database, UI, HTTP client, and so on."
- **p.8 — "The Clean Architecture."** The stability pyramid (Entities → Use Cases → Controllers/Gateways/Presenters → Devices/Web/DB/External Interfaces/UI; axis "Level": "Abstract, General, Rarely Change" ↓ "Concrete, Specific, Change Frequently") beside the dark flow diagram (dashed "Data Flow" arrow through UI → Presenter → Use case → Entity → Repository → Data source; solid "Dependency rule" arrows pointing at the Entity).
- **p.9 — Figure 1.1.** Static cast: User ↔ Delivery Mechanism ‖ (double-line boundary) Boundary `<I>` ×2, Interactor, Entity ×3. Bullet definitions of Interactor, Entity, Boundary `<I>`, Delivery Mechanism (quoted in Concept 24).
- **p.10 — Figure 1.2.** Steps "1-", "2-", "3-": button click; Request Model of "only primitives types"; passed through the input boundary which the interactor implements.
- **p.11 — Figure 1.3.** Step "4-": interactor "controls the dance of the entities (ex: creates the order, modify the customer, etc)"; red dashed arrows interactor → entities.
- **p.12 — Figure 1.4.** Step "5-": the dance "in the opposite direction", gathering results into the Response Model; blue dashed arrows entities → interactor.
- **p.13 — Figure 1.5.** Step "6-": response model out through the output boundary to the delivery mechanism, "and somehow it's delivered to the user."
- **p.14 — Figure 1.6.** Full assembly of the cycle with Controller (red request path) and Presenter → View Model → View (blue response path) replacing the generic Delivery Mechanism.
- **p.15 — Definitions.** Response Model / Presenter / View Model / View, verbatim quotes in Concept 9 and Concept 25, including "(You can test it with your eyes)."
- **p.16 — "What about the Database."** The Martin quote: "If something changes a lot, it should be a plug-in. If something doesn't change very often, it should be plugged into." —Robert C. Martin.
- **p.17 — "What about the Database?"** Three bullets: detail not centre; plug-in to the business rules; no business rules in stored procedures.
- **p.18 — Figure 1.7.** Interactor → Entity Gateway `<I>`; Entity Gateway Implementation below the horizontal double line realizes the interface (arrow up) and uses the Database API; Entities above the line.

- **p.19 — Figure 1.8.** The complete assembled diagram: Controller / Presenter / View Model / View | boundaries + Request/Response Models | Interactor + Entities + Entity Gateway `<I>` | Entity Gateway Implementation + Database API below the line.
- **p.20 — "References."** Seven web sources (listed verbatim under Source Map → Deck reference lists).

OOPrinciples.pdf — all 30 slides

- **p.1 — Title.** "Object-Oriented Principles — Jan Sørensen" over a chalkboard-formula background.
- **p.2 — "History."** "80% of software projects fails"; "The theory of SOLID principles was introduced by Robert C. Martin in his 2000 paper 'Design Principles and Design Patterns'"; "The SOLID acronym itself was introduced later by Michael Feathers."
- **p.3 — "Definition."** The five one-liners, with the S/O/L/I/D initials typeset large: SRP "A class should have only a single responsibility (i.e. changes to only one part of the software's specification should be able to affect the specification of the class)"; OCP "software entities … should be open for extension, but closed for modification"; LSP "objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program"; ISP "many client-specific interfaces are better than one general-purpose interface"; DIP one should "depend upon abstractions, [not] concretions."
- **p.4 — "Single Responsibility Principle."** "A class should have only one reason to change. — Robert C. Martin", illustrated by a toolbox fanned out into individual tools (one tool, one job); slide art credits "enjoy algorithms".
- **p.5 — "Single Responsibility Example."** Without "S": `UserService` with `changePassword(User)` calling its own `checkAccess(User)`. With "S": `UserService.changePassword` calls `SecurityService.checkAccess(user)`; `SecurityService` holds `static boolean checkAccess(User user)`.
- **p.6 — "Open Closed Principle."** Three bullets: open for extension / closed for modification; "add new features … without having to modify our set of preexisting classes"; reduce coupling "to the abstract level — Instead of creating relationships between two concrete classes, we create relationships between a concrete class and an abstract class or an interface."
- **p.7 — "Open/Close Principle Example."** Without "O": `LoanApprovalHandler.approve(PersonalLoanValidator validator)` — the handler's parameter type is the *concrete* validator; `PersonalLoanValidator.isValid()` holds the validation logic. With "O": `interface Validator { boolean isValid(); }`; `PersonalLoanValidator implements Validator`; `HomeLoanValidator implements Validator`; `approve(Validator validator)`.
- **p.8 — "Liskov Substitution Principle (LSP)."** "Subclasses should be substitutable for their base classes."; "Also called the substitutability principle."
- **p.9 — "Liskov Substituion Example"** (sic — missing "t" in the title). Without "L": `Bird { public void fly(){} public void eat(){} }`, `Sparrow extends Bird {}`, `Ostrich extends Bird { fly(){ throw new UnsupportedOperationException(); } }`. With "L": `Bird { void eat(){} }`, `FlightBird extends Bird { void fly(){} }`, `NonFlightBird extends Bird {}`, `Sparrow extends FlightBird {}`, `Ostrich extends NonFlightBird {}`.
- **p.10 — "Interface Segregation Principle (ISP)."** "Many specific interfaces are better than a single, general interface."; "Any interface we define should be highly cohesive."
- **p.11 — "Interface Segregation Example."** Without "I": `interface IUser { changePassword(); checkUserRole(); assignRole(); }`. With "I": `IUser { changePassword(); }`, `IRole { assignRole(); }`, `IUserRole { checkUserRole(); }`.
- **p.12 — "Dependency Inversion Principle."** "Depend upon abstractions. Do not depend upon concretions."; DIP "formalizes the concept of abstract coupling and clearly states that we should couple at the abstract level, not at the concrete level."; "DIP tells us how we can adhere to OCP."

- **p.13 — "Dependency Inversion Example."** Without "D": `Class Payments { CreditCardPaymentMethod ccpm = new CreditCardPaymentMethod(); void makePayments(){ ccpm.transact(); } }`. With "D": `Class Payments { PaymentMethod pm; Payments(PaymentMethod pm_provided) { pm = pm_provided; } void makePayments(){ pm.transact(); } }` (constructor injection; the slide capitalises `Class`).
- **p.14 — "Composite Reuse Principle (CRP)."** "Favor polymorphic composition of objects over inheritance."; "One of the most catastrophic mistakes that contribute to the demise of an object-oriented system is to use inheritance as the primary reuse mechanism."; "Delegation can be a better alternative to Inheritance."
- **p.15 — "Principle of Least Knowledge (PLK)."** "For an operation O on a class C, only operations on the following objects should be called: itself, its parameters, objects it creates, or its contained instance objects."; "Also called the Law of Demeter."; "The basic idea is to avoid calling any methods on an object where the reference to that object is obtained by calling a method on another object (Transitive Visibility)."; "The primary benefit is that the calling method doesn't need to understand the structural makeup of the object it's invoking methods upon."
- **p.16 — "GRASP."** "Acronym stands for General Responsibility Assignment Software Patterns."; "A set of 9 Patterns introduced as a learning aid by Craig Larman."; "Describe fundamental principles for object-oriented design and responsibility assignment, expressed as patterns."; "The skillful assignment of responsibilities is extremely important in object design."; "Determining the assignment of responsibilities often occurs during the creation of interaction diagrams, and certainly during programming."
- **p.17 — "Reference."** "Larman, C., *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3rd Ed. Prentice-Hall, 2004."
- **p.18 — "GRASP: Patterns."** The numbered list: 1. Information Expert, 2. Creator, 3. Low Coupling, 4. High Cohesion, 5. Controller, 6. Polymorphism, 7. Indirection, 8. Pure Fabrication, 9. Protected Variations.
- **p.19 — "GRASP: Information Expert."** "As a general principle of assigning responsibilities to objects, assign a responsibility to the information expert: i.e. the class that has the *information* necessary to fulfill the responsibility."
- **p.20 — "GRASP: Creator."** The five conditions (aggregates / contains / records / closely uses / has the initializing data, "thus B is an Expert with respect to creating A"), the conclusion "B is a *creator* of A objects," and the tie-break "If more than one option applies, prefer a class B which *aggregates* or *contains* class A."
- **p.21 — "GRASP: Low Coupling."** "Assign a responsibility so that coupling remains low," plus the three problems of high coupling: "Changes in related classes force local changes; Harder to understand in isolation; Harder to reuse because its use requires the additional presence of the classes on which it is dependent."
- **p.22 — "GRASP: High Cohesion."** "Assign a responsibility so that cohesion remains high"; low-cohesion classes "do many unrelated things, or do too much work" and are "hard to comprehend, hard to reuse, hard to maintain, Delicate: constantly affected by change."
- **p.23 — "GRASP: Controller."** Façade controller / use-case-or-session controller choices, the same-controller-per-scenario rule, the session definition, and the exclusion list (window/applet/widget/view/document delegate to a controller). Quoted in Concept 25.
- **p.24 — "GRASP: Polymorphism."** "When related alternatives or behaviors vary by type (class), assign responsibility for the behavior — using polymorphic operations — to the types for which the behavior varies."; "Define the behavior in a common base class or, preferably, in an interface."
- **p.25 — "GRASP: Indirection."** "Assign the responsibility to an intermediate object to mediate between other components or services so that they are not directly coupled."; "The intermediary creates an

indirection between the other components."; "Beware of transitive visibility."
- **p.26 — "GRASP: Pure Fabrication."** "Assign a highly cohesive set of responsibilities to an artificial or convenience class that does not represent a problem domain concept — something made up, to support high cohesion, low coupling, and reuse."; example: "a class that is solely responsible for saving objects in some kind of persistent storage medium, such as a relational database; call it the *PersistentStorage*. This class is a Pure Fabrication — a figment of the imagination."
- **p.27 — "GRASP: Protected Variations."** "Identify points of predicted variation or instability; assign responsibilities to create a stable interface around them."; "Note: The term 'interface' is used in the broadest sense of an access view; it does not literally only mean something like a Java or COM interface."
- **p.28–30 — blank.** Three empty trailing slides; no content.

ActualizationLab.pdf — the handout in full

The one-page handout `[ActLab]`, titled "[ActLab] Actualization Lab" by **Jan Corfixen Sørensen**, contains exactly three blocks:

- **Introduction:** "The actualization phase consists of the implementation of the new functionality, its incorporation into the old code, and change propagation that seeks out and updates all places in the old code that require secondary modification."
- **Objectives:** "Understand and explain Clean Architecture in context of Actualization" and "Understand and explain Clean Code Principles in context of Actualization."
- **Portfolio Work:** "Provide examples of the SOLID principles in context of the CASE study." and "Explain Clean Architecture in context of the CASE Study." (A third bullet on the handout is left empty.)

Note what the handout *implies*: the assessed skill is not reciting SOLID or the circle diagram in isolation — it is binding both to the actualization vocabulary (incorporation, ripple, propagation, moment of truth) and to concrete CASE-study classes. The Per-principle example bank (JHotDraw Connection section) is pre-assembled for exactly this.

Compare & Contrast Tables

SOLID at a glance

Principle	Slide wording (core)	Violation smell	Fix	Deck example	Source
SRP	"A class should have only one reason to change"	One class serving two stakeholders/policies	Extract the second responsibility into its own class	`UserService` → `UserService` + `SecurityService`	`(OOPrinciples p.4–5)`
OCP	"Open for extension but closed for modification"; couple "to the abstract level"	Editing a tested class for every new variant; type-switch chains	Introduce an interface; add variants as new implementations	`Validator` ← `PersonalLoanValidator`, `HomeLoanValidator`; `LoanApprovalHandler` untouched	`(OOPrinciples p.6–7)`

Principle	Slide wording (core)	Violation smell	Fix	Deck example	Source
LSP	"Subclasses should be substitutable for their base classes"	Subclass throws/weakens/demands more than the base contract	Restructure so the capability lives only on subtypes that have it	`Ostrich.fly()` throws → `FlightBird` / `NonFlightBird` split	`(OOPrinciples p.8–9)`
ISP	"Many specific interfaces are better than a single, general interface"; interfaces "highly cohesive"	Implementers stubbing methods; clients using a fraction of an interface	Split the fat interface along client roles	`IUser` → `IUser` + `IRole` + `IUserRole`	`(OOPrinciples p.10–11)`
DIP	"Depend upon abstractions. Do not depend upon concretions"; "DIP tells us how we can adhere to OCP"	`new`-ing a concrete collaborator inside the client	Depend on the abstraction; inject the concrete (constructor injection)	`Payments` + injected `PaymentMethod` instead of `new CreditCardPaymentMethod()`	`(OOPrinciples p.12–13)`

The supporting principles vs. their look-alikes

Pair	The difference that matters	Sources
CRP vs. inheritance-based OCP	OCP's deck example still *uses* inheritance/implementation (new `Validator` implementations); CRP warns against inheritance as the **primary reuse** mechanism and prefers *polymorphic composition* + delegation. Adding a variant = OCP territory; reusing behaviour = CRP territory.	`(OOPrinciples p.6–7, p.14)`
PLK vs. Indirection	PLK is the *rule* (talk only to self, parameters, creations, fields; avoid transitive visibility); Indirection is the *pattern* that restores compliance by inserting a mediator. Indirection's own slide warns "Beware of transitive visibility" — a careless mediator just adds a link to the train-wreck.	`(OOPrinciples p.15, p.25)`
SRP vs. High Cohesion	Same instinct at different vocabulary levels: SRP is the SOLID statement ("one reason to change"), High Cohesion is the GRASP *evaluative* pattern ("does many unrelated things / too much work" = low cohesion). Cite SRP for class design, High Cohesion when judging a responsibility assignment.	`(OOPrinciples p.4, p.22)`
Low Coupling vs. DIP	Low Coupling is the *goal/yardstick* (fewer dependencies = less forced local change); DIP is one *mechanism* (point the remaining dependencies at abstractions). You can satisfy DIP and still be over-coupled to too many abstractions.	`(OOPrinciples p.21, p.12)`
Pure Fabrication vs. Information Expert	Expert is the default (put behaviour with the data); Fabrication is the licensed exception (invent a non-domain class when Expert would wreck a domain class's cohesion — e.g. `PersistentStorage`).	`(OOPrinciples p.19, p.26)`

Mapping SOLID ↔ GRASP ↔ Clean Architecture ↔ Actualization

Idea	SOLID form	GRASP form	Clean Architecture form	Actualization payoff
Add behaviour without editing	OCP `(OOPrinciples p.6)`	Polymorphism `(OOPrinciples p.24)`	New use case / new adapter plugs in	Pure new code; minimal ripple (`Pig (Actualization p.6–7)`)

Idea	SOLID form	GRASP form	Clean Architecture form	Actualization payoff
Point dependencies at stability	DIP `(OOPrinciples p.12)`	Protected Variations `(OOPrinciples p.27)`	Dependency Rule `(Clean Architecture p.8)`	Volatile-side changes can't reach stable code
Keep concerns apart	SRP `(OOPrinciples p.4)`	High Cohesion `(OOPrinciples p.22)`	Layer separation (Entities vs. Use Cases vs. Adapters) `(Clean Architecture p.7)`	A change request names few classes; better recall
Narrow the contracts	ISP `(OOPrinciples p.10)`	Low Coupling `(OOPrinciples p.21)`	Boundary `<I>` per use-case direction `(Clean Architecture p.9)`	Signature changes ripple to fewer clients
Decouple via a middleman	—	Indirection `(OOPrinciples p.25)`	Entity Gateway / Repository `(Clean Architecture p.18)`	DB/UI swaps actualized in one implementation class
Keep persistence out of the domain	—	Pure Fabrication `(OOPrinciples p.26)`	Gateway implementation below the line `(Clean Architecture p.18)`	"Swap SQL→NoSQL" never reaches entities `(Clean Architecture p.4)`

The three incorporation shapes vs. expected ripple

Shape	Wiring on the slide	Who depends on whom	Expected propagation	Source
Local	New class stands alone; no boundary-crossing edges drawn	Old code will merely invoke the newcomer	Minimal — nothing in old code structurally re-wired	`(Actualization p.9)`
Composite	New class composes **old bottom-layer classes** (two filled diamonds crossing the boundary)	New → old (the new class consumes existing suppliers)	Moderate — the new cluster must be wired to its old parts	`(Actualization p.10)`
New supplier	Old top class aggregates the new class **and** the new class composes old classes	Old → new **and** new → old	Largest — every old client of the new supplier may need updating	`(Actualization p.11)`
(Replacement)	Like supplier, plus an old class crossed out	New class takes over the deleted class's role and edges	Supplier-level ripple **plus** deletion propagation (all references removed)	`(Actualization p.15, p.22)`

Precision vs. Recall side by side

	Precision	Recall
Formula	TP / (TP + FP)	TP / (TP + FN)
Denominator counts	Everything you **predicted** would change	Everything that **actually** changed
Failure it measures	False alarms (flagged but unchanged)	Misses (changed but unflagged)
Ericsson value	30/(30+0) = **100%**	30/(30+64) = **32%**
Which is hard	Easy — don't over-flag	Hard — invisible dependencies hide the FNs
Cost of failure	Wasted inspection effort	Blown schedules; "missed the other two thirds!"
Source	`(Actualization p.26)`	`(Actualization p.27)`

Architecture vs. design pattern

	Design pattern	Software architecture
Slide definition	"the re-usable form of a solution to a design problem"	"the high level structures of a software system and the discipline of creating such structures and systems"
Scope	Local — a few collaborating classes	Global — the whole system's partitioning and dependency regime
Slide examples	Singleton, Observer, MVC, MVVM, MVP	Hexagonal, Onion, **The Clean architecture**
In this lecture	Presenter (MVP), ViewModel (MVVM), Repository/Gateway appear *inside* the architecture as patterns `(Clean Architecture p.7)`	The concentric layers + Dependency Rule `(Clean Architecture p.5–8)`
Source	`(Clean Architecture p.2)`	`(Clean Architecture p.2)`

Easily confused distinctions (rapid-fire)

Confusable pair	The one-line discriminator	Sources
Incorporation vs. change propagation	Incorporation = the deliberate primary plug-in; propagation = the forced secondary edits afterwards	`(Actualization p.8, p.16–21)`
Impact analysis vs. change propagation	Prediction before coding vs. the actual walk — propagation is the "moment of truth" that scores the prediction	`(Actualization p.23)`
Ripple effect vs. change propagation	Ripple = the phenomenon (change spreads); propagation = the activity of chasing it to consistency	`(Actualization p.5)`; `[ActLab]`
Small vs. larger change	Direct edit in old code vs. separate new classes + incorporation	`(Actualization p.2–5)`
Entities vs. Use Cases	Enterprise-wide ("least likely to change") vs. application-specific rules ("isolated from … database, common frameworks, and the UI")	`(Clean Architecture p.7)`
Request Model vs. Response Model	Inbound, "only primitives types", built by the delivery mechanism vs. outbound, built by the interactor, "not presentable"	`(Clean Architecture p.10, p.15)`
Response Model vs. View Model	Domain-typed result (date object, money object) vs. display-ready strings/booleans written by the Presenter	`(Clean Architecture p.15)`
Presenter vs. View	Translates response → view model (testable logic) vs. "grabbing the data … and put it on the screen" (no logic, eyes-only testing)	`(Clean Architecture p.15)`
Gateway interface vs. Gateway implementation	Declared with the business rules vs. living below the boundary line beside the Database API	`(Clean Architecture p.18)`
Dependency Rule vs. DIP	Same idea at two scales: architecture rings vs. class-level coupling; "DIP tells us how we can adhere to OCP"	`(Clean Architecture p.8)`; `(OOPrinciples p.12)`
GRASP Polymorphism vs. OCP	GRASP names *where* the responsibility goes (the varying types); OCP names the *resulting property* (closed to modification)	`(OOPrinciples p.24, p.6)`
GRASP Controller vs. Clean-Architecture Controller	GRASP's is the responsibility-assignment rule (non-UI class handles system events); the deck's Figure 1.6 Controller is that rule realised at the boundary	`(OOPrinciples p.23)`; `(Clean Architecture p.14)`

Confusable pair	The one-line discriminator	Sources
Precision vs. recall	Divide by predicted-changed vs. divide by actually-changed	`(Actualization p.26–27)`
True negative vs. false negative	Predicted-unchanged & unchanged (42) vs. predicted-unchanged & changed (64 — the dangerous cell)	`(Actualization p.24–25)`

Self-Test Q&A (Exam Drill)

Answers cite the slide that grades them. Cover the deck you are weakest on first.

Actualization process

1. *Q: Define the actualization phase in one sentence.* A: The phase in which the new functionality is implemented, incorporated into the old code, and change-propagated until all secondary modifications are done `[ActLab]` `(Actualization p.1, p.5)`.
2. *Q: What single factor makes the actualization process vary?* A: The **size** of the change `(Actualization p.1)`.
3. *Q: How are small changes performed, and what is the deck's example?* A: Directly in the old code; widening `Address`'s `zip[5]` to `zip[9]` `(Actualization p.2–4)`.
4. *Q: Name the three bullet steps of a larger change.* A: Implement the new classes separately; plug the new code in (incorporation); the change can propagate (ripple effect) `(Actualization p.5)`.
5. *Q: What does "the composite responsibility of Farm was extended by the concept Pig" mean?* A: `Farm`'s responsibility, realised through the `FarmAnimal` family, grew by a new subtype — without editing `Farm` `(Actualization p.6–7)`.
6. *Q: List the three incorporation shapes and rank them by propagation risk.* A: New responsibility local < composite < incorporating a new supplier `(Actualization p.9–11)`.
7. *Q: In the PoS change, which old class is modified at incorporation and how?* A: `Store` — it gains an aggregation of the new `Cashiers` class (it is the only shaded old class on the slide) `(Actualization p.13–14)`.
8. *Q: What happens during "replacement of a class"?* A: A new class takes over an obsolete class's role and connections; the obsolete class is crossed out and its deletion propagates further `(Actualization p.15, p.22)`.
9. *Q: When does change propagation end?* A: When no marked/inconsistent classes remain — on the slides, when no class is green `(Actualization p.21)`.
10. *Q: Which classes in the taxCategory walk changed, and which were merely visited?* A: Changed: `item`, `saleLineItem`, `sale`; visited-unchanged: `store`, `register` `(Actualization p.16–21)`.
11. *Q: Why does deleting code cause propagation?* A: "All references to the deleted functionality must be deleted — secondary changes propagate to other classes" `(Actualization p.22)`.
12. *Q: Why is change propagation called the moment of truth?* A: It confirms or refutes impact analysis's predictions — and that accuracy "is important for software managers" `(Actualization p.23)`.
13. *Q: Reproduce the Ericsson confusion matrix.* A: Predicted-Unchanged: 42 (actual unchanged), 64 (actual changed); Predicted-Changed: 0 (actual unchanged), 30 (actual changed); total 136 `(Actualization p.24)`.
14. *Q: Compute Ericsson's precision and recall.* A: Precision 30/(30+0) = 100%; recall 30/(30+64) = 32% `(Actualization p.26–27)`.

15. *Q: Which confusion-matrix cell is "the dangerous one" and why?* A: False negatives (64): classes that changed but were never predicted — unplanned work `(Actualization p.25, p.27)`.
16. *Q: Give the deck's three statements about underestimation.* A: Common in software engineering (a consequence of invisibility); makes planning difficult; common in other fields also `(Actualization p.28)`.

Clean Architecture

17. *Q: Define design pattern and software architecture per the deck, with the slide's examples.* A: Pattern = "re-usable form of a solution to a design problem" (Singleton, Observer, MVC, MVVM, MVP); architecture = "high level structures of a software system and the discipline of creating such structures" (Hexagonal, Onion, Clean) `(Clean Architecture p.2)`.
18. *Q: List the four characteristics of a successful software architecture.* A: Testable; independent of the UI; independent of the database; independent of frameworks and external entities `(Clean Architecture p.3)`.
19. *Q: What does "independent of any external agency" mean?* A: "Business rules should know nothing about the outside world" `(Clean Architecture p.4)`.
20. *Q: Name the four layers inside-out with their colour-legend names.* A: Entities (Enterprise Business Rules) → Use Cases (Application Business Rules) → Interface Adapters (Controllers/Presenters/Gateways) → Frameworks & Drivers (Web/DB/Devices/UI/External Interfaces) `(Clean Architecture p.6–7)`.
21. *Q: Which layer contains Presenters, ViewModels and Gateways, and from which patterns do those names come?* A: Interface Adapters; MVP, MVVM, and Gateways "also known as Repositories" `(Clean Architecture p.7)`.
22. *Q: State the Dependency Rule and the pyramid's two pole labels.* A: Source-code dependencies point only inward; poles: "Abstract, General, Rarely Change" (top/Entities) vs. "Concrete, Specific, Change Frequently" (bottom/External Interfaces) `(Clean Architecture p.8)`.
23. *Q: Who implements the input boundary, and who implements the output boundary?* A: The interactor implements the input boundary; output boundaries are "implemented by other objects" (the presenter side) `(Clean Architecture p.9, p.14)`.
24. *Q: What may a Request Model contain?* A: "Only primitives types" — a plain data structure built by the delivery mechanism `(Clean Architecture p.10)`.
25. *Q: Recite steps 4–6 of the use-case cycle.* A: The interactor "controls the dance of the entities"; gathers results "in the opposite direction" into the Response Model; the response model exits via the output boundary to the delivery mechanism and is delivered to the user `(Clean Architecture p.11–13)`.
26. *Q: Why is the Response Model "not presentable"?* A: Its data are still domain objects — "a date ... would be a date object, and the currency would be a money object"; formatting is the Presenter's job `(Clean Architecture p.15)`.
27. *Q: What belongs in a View Model?* A: Display-ready data: the button-label string, the boolean for whether the button is active/grey, menu-item names and their order `(Clean Architecture p.15)`.
28. *Q: How dumb should the View be?* A: "So stupid so you don't have to test it. (You can test it with your eyes.)" `(Clean Architecture p.15)`.
29. *Q: Quote Martin's plug-in rule.* A: "If something changes a lot, it should be a plug-in. If something doesn't change very often, it should be plugged into." `(Clean Architecture p.16)`.
30. *Q: Give the three database theses.* A: The database is a detail, not the centre; it should be a plug-in to the business rules; business rules shouldn't be written in stored procedures `(Clean Architecture p.17)`.
31. *Q: In Figure 1.7, where is the gateway interface declared and where implemented?* A: Declared with the business rules (above the boundary line, depended on by the interactor); implemented below the line by

the Entity Gateway Implementation, which uses the Database API `(Clean Architecture p.18)`.

OO Principles & GRASP

32. *Q: Who introduced SOLID theory, when, in what paper — and who coined the acronym?* A: Robert C. Martin, 2000, "Design Principles and Design Patterns"; acronym by Michael Feathers `(OOPrinciples p.2)`.
33. *Q: Which statistic opens the History slide?* A: "80% of software projects fails" `(OOPrinciples p.2)`.
34. *Q: Recite all five SOLID one-liners.* A: See `(OOPrinciples p.3)` — single responsibility; open for extension/closed for modification; subtypes replaceable without altering correctness; many client-specific interfaces; depend upon abstractions, not concretions.
35. *Q: What is the SRP fix in the deck's example?* A: Move `checkAccess` out of `UserService` into a `SecurityService` so password policy and access policy each have one reason to change `(OOPrinciples p.5)`.
36. *Q: How does the OCP example free `LoanApprovalHandler`?* A: `approve` takes the `Validator` interface instead of the concrete `PersonalLoanValidator`, so new validators (e.g. `HomeLoanValidator`) are added without modifying the handler `(OOPrinciples p.7)`.
37. *Q: Why does `Ostrich extends Bird` violate LSP, and what is the fix?* A: It throws `UnsupportedOperationException` on `fly()`, so it is not substitutable; split into `FlightBird` / `NonFlightBird` `(OOPrinciples p.9)`.
38. *Q: What two properties should interfaces have per ISP?* A: Many and client-specific rather than single and general; highly cohesive `(OOPrinciples p.10)`.
39. *Q: Which principle "tells us how we can adhere to OCP"?* A: DIP `(OOPrinciples p.12)`.
40. *Q: What technique does the DIP example use to supply the concretion?* A: Constructor injection — `Payments(PaymentMethod pm_provided)` `(OOPrinciples p.13)`.
41. *Q: Quote CRP's "catastrophic" warning.* A: "One of the most catastrophic mistakes that contribute to the demise of an object-oriented system is to use inheritance as the primary reuse mechanism" `(OOPrinciples p.14)`.
42. *Q: State PLK's allowed call targets.* A: Itself, its parameters, objects it creates, its contained instance objects `(OOPrinciples p.15)`.
43. *Q: What is transitive visibility?* A: Calling methods on an object whose reference was obtained by calling a method on another object — the thing PLK avoids and Indirection warns about `(OOPrinciples p.15, p.25)`.
44. *Q: Expand GRASP and name its author and source.* A: General Responsibility Assignment Software Patterns; Craig Larman, *Applying UML and Patterns*, 3rd Ed., Prentice-Hall, 2004 `(OOPrinciples p.16–17)`.
45. *Q: When does responsibility assignment happen?* A: "Often ... during the creation of interaction diagrams, and certainly during programming" `(OOPrinciples p.16)`.
46. *Q: List the nine GRASP patterns in slide order.* A: Information Expert, Creator, Low Coupling, High Cohesion, Controller, Polymorphism, Indirection, Pure Fabrication, Protected Variations `(OOPrinciples p.18)`.
47. *Q: Creator names five conditions — list them and the tie-break.* A: B aggregates A; B contains A; B records instances of A; B closely uses A; B has A's initializing data; tie-break: prefer the aggregator/container `(OOPrinciples p.20)`.
48. *Q: Name three problems of a highly coupled class.* A: Changes in related classes force local changes; harder to understand in isolation; harder to reuse (requires the presence of its dependencies) `(OOPrinciples p.21)`.

49. *Q: Name the four problems of a low-cohesion class.* A: Hard to comprehend, hard to reuse, hard to maintain, delicate (constantly affected by change) `(OOPrinciples p.22)`.
50. *Q: Which classes must never be GRASP Controllers?* A: "Window," "applet," "widget," "view," and "document" classes — they delegate system events to a controller `(OOPrinciples p.23)`.
51. *Q: Where should polymorphic behaviour be defined, preferably?* A: "In a common base class or, preferably, in an interface" `(OOPrinciples p.24)`.
52. *Q: Give Pure Fabrication's canonical example and its punchline.* A: `PersistentStorage`, a class solely responsible for saving objects to persistent storage — "a figment of the imagination" `(OOPrinciples p.26)`.
53. *Q: In Protected Variations, what does "interface" mean?* A: "The broadest sense of an access view — not literally only … a Java or COM interface" `(OOPrinciples p.27)`.

Synthesis (lab-style)

54. *Q: Use the taxCategory walk to argue why Protected Variations saves money.* A: `sale` changed only because `saleLineItem` changed because `item` changed `(Actualization p.16–21)`; a stable interface at any link `(OOPrinciples p.27)` stops the walk earlier, so fewer secondary modifications and better effort predictability `(Actualization p.23)`.
55. *Q: Connect the Entity Gateway to a concrete change request.* A: "Switch RDBS to NoSQL" `(Clean Architecture p.4)` is actualized entirely inside the Entity Gateway Implementation; the interactor depends only on the unchanged gateway interface `(Clean Architecture p.18)` — propagation cannot cross the boundary line.
56. *Q: Which SOLID principle most directly explains the `Pig` slide, and why?* A: OCP — the new behaviour arrives as a new subtype while `Farm` and existing clients stay closed to modification `(Actualization p.6–7)` `(OOPrinciples p.6)`; LSP is the safety condition that makes it work `(OOPrinciples p.8)`.

Lab Walkthrough & Portfolio Strategy

What the lab demands, item by item

- **Objective 1 — "Understand and explain Clean Architecture in context of Actualization"** `[ActLab]`. The connective argument to rehearse: actualization cost = incorporation effort + propagation reach `(Actualization p.5, p.8)`; Clean Architecture pre-positions stable seams (boundaries, gateways, the Dependency Rule `(Clean Architecture p.8–9, p.18)`) so that whole categories of change request — new UI, new DB, new framework `(Clean Architecture p.3–4)` — incorporate in the outer rings and propagate no further. Use the plug-in quote `(Clean Architecture p.16)` as the thesis sentence.
- **Objective 2 — "Understand and explain Clean Code Principles in context of Actualization"** `[ActLab]`. Map each principle to its propagation effect: SRP/High Cohesion shrink the per-request class count `(OOPrinciples p.4, p.22)`; OCP/Polymorphism/DIP turn modifications into additions `(OOPrinciples p.6, p.12, p.24)`; LSP keeps the additions from breaking clients `(OOPrinciples p.8)`; ISP/Low Coupling narrow how far a signature change travels `(OOPrinciples p.10, p.21)`; PLK eliminates the transitive dependencies that destroy impact-analysis recall `(OOPrinciples p.15)` `(Actualization p.27–28)`.
- **Portfolio item 1 — SOLID examples from the CASE study** `[ActLab]`: use the Per-principle CASE-study example bank (JHotDraw Connection section); each entry already follows the required principle-element-payoff pattern.

- **Portfolio item 2 — Clean Architecture on the CASE study** `[ActLab]`: assign the CASE study's classes to the four rings `(Clean Architecture p.6–7)`, draw the Dependency Rule, then walk one concrete change request through Figures 1.6/1.7 vocabulary (controller, boundary, interactor, response model, presenter, gateway) `(Clean Architecture p.14, p.18)`.

Marking-friendly structure for both portfolio answers

1. **Definition with citation** — quote the slide-exact wording (the lab examiner wrote these decks).
2. **CASE-study element** — name an actual class/interface, not a category.
3. **Actualization tie-in** — one sentence on incorporation shape `(Actualization p.9–11)` or propagation reach `(Actualization p.16–21)`.
4. **Counterfactual** — what the ripple would look like *without* the principle (the Ericsson 32%-recall world `(Actualization p.27)`).

Annotated Code Listings from the Slides

Every code fragment the three decks show, collected in one place with the reading notes an examiner expects you to be able to produce. (Transcribed as printed, including the decks' casing quirks.)

Address — the small-change pair (Actualization p.2–4)

Before `(Actualization p.2)`:

```
class Address
{
public move();
protected String name;
protected String streetAddress;
protected String city;
protected char state[2], zip[5];
};
```

After `(Actualization p.3, repeated unchanged on p.4)`:

```
class Address
{
public move();
protected String name;
protected String streetAddress;
protected String city;
protected char state[2], zip[9];
};
```

Reading notes. (1) The diff is a single token: `zip[5]` → `zip[9]` — widening the US ZIP field from 5 to 9 characters (ZIP+4). (2) Everything else — `move()`, `name`, `streetAddress`, `city`, `state[2]` — is untouched: that *is* the definition of a small change "done directly in old code" `(Actualization p.2)`. (3) No new class, no incorporation, and the deck spends a third slide on the identical after-state to stress that the change *stays* local. (4) If asked "when is editing old code directly acceptable?", this is the canonical answer-by-example.

Pig — polymorphic extension (Actualization p.7)

```
class Pig : public FarmAnimal
{
public:
void makeSound() {cout<<"Oink";}
};
```

Reading notes. (1) Public inheritance from `FarmAnimal` plus one overridden operation is the *entire* change — no existing file is edited. (2) The slide's conclusion: "Farm now can declare objects of the type Cow, Sheep, or Pig — the composite responsibility of Farm was extended by the concept Pig" `(Actualization p.7)`. (3) This is the actualization deck's only inheritance-based incorporation; contrast with the composition diamonds of `(Actualization p.9–11)`. (4) Cite it as the lived example of OCP `(OOPrinciples p.6)` and GRASP Polymorphism `(OOPrinciples p.24)`.

UserService / SecurityService — SRP pair (OOPrinciples p.5)

Without "S":

```
class UserService {
  void changePassword(User user) {
    if (checkAccess(user)) { /* Grant option to change */ }
  }
  boolean checkAccess(User user) { /* Verify if the user is valid. */ }
}
```

With "S":

```
class UserService {
  void changePassword(User user) {
    if (SecurityService.checkAccess(user)) { /* Grant option to change */ }
  }
}
class SecurityService {
  static boolean checkAccess(User user) { /* check the access. */ }
}
```

Reading notes. (1) The violation is *cohabitation*: password management and access verification are two specifications that change for different reasons, held in one class `(OOPrinciples p.4)`. (2) The fix is extraction; the deck makes `checkAccess` `static` in `SecurityService` — the call site changes from `checkAccess(user)` to `SecurityService.checkAccess(user)`. (3) After the split, a change to access policy touches `SecurityService` only — the propagation argument in miniature.

Validator / LoanApprovalHandler — OCP pair (OOPrinciples p.7)

Without "O":

```
class LoanApprovalHandler {
  void approve(PersonalLoanValidator validator) {
    if ( validator.isValid()) {
      //Process the loan.
    }
  }
}
class PersonalLoanValidator {
  boolean isValid() {
```

```
    //Validation logic
  }
}
```

With "O":

```
interface Validator { boolean isValid(); }
class PersonalLoanValidator implements Validator {
  boolean isValid() {}
}
class HomeLoanValidator implements Validator {
  boolean isValid() {}
}
class LoanApprovalHandler {
  void approve(Validator validator) {
    if ( validator.isValid()) {
      //Process the loan.
    }
  }
}
```

Reading notes. (1) The violation lives in the **parameter type**: `approve(PersonalLoanValidator …)` welds the handler to one concrete validator, so every new loan type means editing the handler. (2) The fix introduces the abstraction and retypes the parameter: `approve(Validator …)` — the handler's body is unchanged. (3) This is the slide-level proof of the OCP tenet "instead of creating relationships between two concrete classes, we create relationships between a concrete class and an abstract class or an interface" `(OOPrinciples p.6)`, and the class-scale version of the Boundary `<I>` move `(Clean Architecture p.9)`.

Bird hierarchy — LSP pair (OOPrinciples p.9)

Without "L":

```
class Bird {
  public void fly(){}
  public void eat(){}
}
class Sparrow extends Bird {}
class Ostrich extends Bird{
  fly(){
    throw new UnsupportedOperationException();
  }
}
```

With "L":

```
class Bird {
  void eat(){}
}
class FlightBird extends Bird {
  void fly(){}
}
class NonFlightBird extends Bird {}

class Sparrow extends FlightBird {}
class Ostrich extends NonFlightBird {}
```

Reading notes. (1) The broken version compiles — the violation is *behavioural*: any client holding a `Bird` and calling `fly()` blows up when handed an `Ostrich`. (2) The repair moves `fly()` *down* to a `FlightBird` intermediate class, so the capability exists only where it is genuinely supported; `Bird` keeps the universally valid `eat()`. (3) Pattern to memorise: *don't inherit a capability you must cancel — restructure so the capability enters the hierarchy at the right level.*

IUser segregation — ISP pair (OOPrinciples p.11)

Without "I":

```
interface IUser{
 changePassword();
 checkUserRole();
 assignRole();
}
```

With "I":

```
interface IUser{
 changePassword();
}
interface IRole{
 assignRole();
}
interface IUserRole{
 checkUserRole();
}
```

Reading notes. (1) Three unrelated client roles (self-service password change, role administration, role checking) were trapped in one contract. (2) The fix yields three single-method interfaces — each "highly cohesive" `(OOPrinciples p.10)`; an implementer or client of one is no longer coupled to the other two. (3) Note the deck's split assigns `checkUserRole()` to `IUserRole` and `assignRole()` to `IRole` — keep the method-to-interface mapping straight when reproducing it.

Payments / PaymentMethod — DIP pair (OOPrinciples p.13)

Without "D":

```
Class Payments {
CreditCardPaymentMethod ccpm = new CreditCardPaymentMethod();
void makePayments(){
 ccpm.transact();
}
}
```

With "D":

```
Class Payments {
PaymentMethod pm;
Payments(PaymentMethod pm_provided) {
 pm = pm_provided;
}
void makePayments(){
 pm.transact();
```

```
}
}
```

Reading notes. (1) The violation is the inline `new CreditCardPaymentMethod()` — `Payments` chooses *and* constructs its concrete collaborator, coupling at the concrete level `(OOPrinciples p.12)`. (2) The fix changes the field type to the abstraction (`PaymentMethod`) and accepts the instance through the constructor (`pm_provided`) — **constructor injection**, no framework involved. (3) `makePayments()` is byte-for-byte the same except `ccpm` → `pm`: inversion changes *wiring*, not logic. (4) The deck prints `Class` with a capital C on both slides — transcription artefact, not Java. (5) Exam link: "DIP tells us how we can adhere to OCP" `(OOPrinciples p.12)` — after this refactor, new payment methods are pure extension.

Mock Exam Questions with Model Answers

Long-form questions in the style the lab's objectives imply, with model answers built only from slide content.

Q1. "Describe the actualization phase and explain how the process differs with the size of the change." (10 marks)

Model answer. Actualization is the phase of the software change process in which "programmers implement the new functionality according to the change request" `(Actualization p.1)`; per the lab definition it "consists of the implementation of the new functionality, its incorporation into the old code, and change propagation that seeks out and updates all places in the old code that require secondary modification" `[ActLab]`. It follows Initiation, Concept Location, Impact Analysis and Prefactoring, precedes Postfactoring and Conclusion, and runs under continuous Verification `(Actualization p.1)`. The process "varies — depends on the size of the change" `(Actualization p.1)`. **Small changes** are made directly in the old code: the deck widens `Address`'s `zip[5]` to `zip[9]` — one field, one class, no new structure `(Actualization p.2–4)`. **Larger changes** are implemented as new classes "separately from the old code", then "plugged into the existing code (incorporation)", after which "the change can propagate to other components of the system (ripple effect)" `(Actualization p.5)`. Incorporation can take three structural shapes — the new responsibility may be local (self-contained), composite (the new class aggregates existing classes), or a new supplier (existing clients come to depend on the new class) `(Actualization p.9–11)` — and the shape predicts how much propagation follows. Marks are typically reserved for: the three-part definition, the phase context, the ZIP example, the separate-then-plug-in discipline, and at least one incorporation shape.

Q2. "Using the Ericsson Radio Systems data, explain how the accuracy of impact analysis is measured and what the results imply." (10 marks)

Model answer. After change propagation completes, the *actual* set of changed classes is known, and propagation acts as "the moment of truth: it confirms or refutes the predictions of impact analysis" `(Actualization p.23)`. Prediction quality is scored with information-retrieval measures over a confusion matrix `(Actualization p.24–26)`. At Ericsson (136 classes total), programmers predicted 30 classes would change that did change (TP = 30), predicted 0 that did not change (FP = 0), left 42 correctly unflagged (TN = 42), and missed 64 that changed unflagged (FN = 64) `(Actualization p.24–25)`. **Precision** = TP/(TP+FP) = 30/30 = **100%** — no false alarms `(Actualization p.26)`. **Recall** = TP/(TP+FN) = 30/94 = **32%** — "programmers estimated that the changes will impact only about a third of all classes that actually changed — missed the other two thirds!" `(Actualization p.27)`. Implications: under-estimation is "common in software engineering", a "consequence of invisibility" of dependencies, and it "makes planning difficult" `(Actualization p.28)`; managers need these accuracy figures because effort plans built on a 32%-recall

impact set will overrun threefold on scope. The asymmetry (perfect precision, poor recall) shows the failure mode is *missing* work, not inventing it.

Q3. "Draw and explain The Clean Architecture, including the Dependency Rule, and explain why the database is 'a detail'." (15 marks)

Model answer. Draw four concentric rings `(Clean Architecture p.6)`: centre **Entities** — "enterprise-wide business rules ... the least likely to change"; then **Use Cases** (interactors) — "application-specific business rules ... isolated from changes to the database, common frameworks, and the UI"; then **Interface Adapters** — "convert data from a convenient format for entities and use cases to a format applicable to databases and the web", including Presenters (MVP), ViewModel (MVVM) and Gateways/Repositories; outermost **Frameworks and Drivers** — "the web framework, database, UI, HTTP client, and so on" `(Clean Architecture p.7)`. Annotate the **Dependency Rule**: source-code dependencies point only inward, toward the "Abstract, General, Rarely Change" apex of the stability pyramid, away from the "Concrete, Specific, Change Frequently" base `(Clean Architecture p.8)` — while *data* flows straight through the layers (UI → Presenter → Use case → Entity → Repository → Data source), a different arrow than the dependencies `(Clean Architecture p.8)`. A successful architecture is thereby "Testable, Independent of the UI, Independent of the Database, Independent of Frameworks and External Entities" `(Clean Architecture p.3)`. The database is a detail because "if something changes a lot, it should be a plug-in. If something doesn't change very often, it should be plugged into" `(Clean Architecture p.16)`: the DB "shouldn't be the center of your architecture", "should be a plug-in to the business rules", and "business rules shouldn't be written in stored procedures" `(Clean Architecture p.17)`. Structurally, the interactor depends on an **Entity Gateway** `<I>` declared with the business rules; the **Entity Gateway Implementation** below the boundary realizes it and talks to the **Database API** `(Clean Architecture p.18)` — so switching "from RDBS to NoSQL or any other" `(Clean Architecture p.4)` is actualized entirely below the line.

Q4. "Trace one use-case execution through the Clean Architecture objects." (10 marks)

Model answer. Follow the deck's six numbered steps `(Clean Architecture p.10–13)`: (1) "User clicks a button (on a web form for example)". (2) The delivery mechanism packs the submitted data "into a data structure which contains only primitives types (Request Model)". (3) "The request model passed through the input boundary and since the interactor implements the input boundary, the interactor received that request model". (4) The "interactor then uses that request model and controls the dance of the entities (ex: creates the order, modify the customer, etc)". (5) It then works "in the opposite direction as it gather[s] up all the results" into the **Response Model**. (6) "The response model is passed back out through the output boundary to the delivery mechanism that implement[s] it and somehow it's delivered to the user". In the assembled Figure 1.6 `(Clean Architecture p.14)`, the delivery mechanism resolves into a **Controller** (request side) and a **Presenter** (response side): the Presenter translates the Response Model's domain objects (date object, money object) into a **View Model** of display-ready strings and booleans, and the **View** does nothing but copy the View Model to the screen — "so stupid so you don't have to test it. (You can test it with your eyes.)" `(Clean Architecture p.15)`. The boundary interfaces make every dependency at the seam point inward while the data crosses both ways.

Q5. "For each SOLID principle, state it and give the lecture's example of a violation and its repair." (15 marks)

Model answer. S — "A class should have only one reason to change" `(OOPrinciples p.4)`: `UserService` both changed passwords and verified access; repair extracts `SecurityService` `(OOPrinciples p.5)`. **O** — entities "open for extension, but closed for modification"; couple at the abstract level `(OOPrinciples p.3, p.6)`: `LoanApprovalHandler.approve(PersonalLoanValidator)` had to be edited per loan type; repair

introduces `interface Validator` with `PersonalLoanValidator` / `HomeLoanValidator` implementations and retypes the parameter `(OOPrinciples p.7)`. **L** — "Subclasses should be substitutable for their base classes" (the substitutability principle) `(OOPrinciples p.8)`: `Ostrich extends Bird` threw `UnsupportedOperationException` on `fly()`; repair splits `FlightBird` / `NonFlightBird` `(OOPrinciples p.9)`. **I** — "Many specific interfaces are better than a single, general interface", each "highly cohesive" `(OOPrinciples p.10)`: fat `IUser` with `changePassword/checkUserRole/assignRole`; repair segregates `IUser` / `IRole` / `IUserRole` `(OOPrinciples p.11)`. **D** — "Depend upon abstractions. Do not depend upon concretions", and "DIP tells us how we can adhere to OCP" `(OOPrinciples p.12)`: `Payments` `new`ed a `CreditCardPaymentMethod`; repair injects a `PaymentMethod` abstraction through the constructor `(OOPrinciples p.13)`. Top marks add the history: Martin's 2000 paper "Design Principles and Design Patterns", acronym by Michael Feathers `(OOPrinciples p.2)`.

Q6. "Pick four GRASP patterns and show how each would have reduced the propagation in the taxCategory example." (12 marks)

Model answer. The walk `(Actualization p.16–21)` modified `item`, then `saleLineItem`, then `sale`, with `store` and `register` inspected but unchanged. **Low Coupling** `(OOPrinciples p.21)`: every red class changed because it "relies on" a changed neighbour — assigning responsibilities to minimise such reliance shortens the chain ("changes in related classes force local changes"). **Protected Variations** `(OOPrinciples p.27)`: had the tax-relevant access to `item` been wrapped in a stable interface at the point of predicted variation, `saleLineItem` (and transitively `sale`) would have depended on the unchanged interface, ending the walk at `item`. **Information Expert** `(OOPrinciples p.19)`: putting tax computation in the class holding the tax data (`taxCategory` / `item`) avoids scattering tax knowledge into `saleLineItem` and `sale` in the first place. **Indirection** `(OOPrinciples p.25)`: an intermediary between `item` and its clients would decouple them ("so that they are not directly coupled"), though one must "beware of transitive visibility" — the mediator must genuinely hide `item`'s structure, or the coupling merely lengthens (PLK `(OOPrinciples p.15)`). Conclusion sentence: each pattern converts secondary modifications into no-ops, which is precisely the recall problem of `(Actualization p.27–28)` becoming smaller.

Source Map

Source	Pages / slides	Topic(s) covered	Controlled-topic tags
`Actualization.pdf [Raj13 Ch.8]`	p.1	Actualization defined; place in process; varies with change size	Software Change Process, Actualization
`Actualization.pdf`	p.2–4	Small changes done directly in old code (Address ZIP `zip[5]` → `zip[9]`)	Actualization
`Actualization.pdf`	p.5	Larger changes: implement separately, incorporate, ripple effect	Actualization
`Actualization.pdf`	p.6–7	Polymorphism as actualization (Farm / FarmAnimal / Pig)	Actualization, OO Principles
`Actualization.pdf`	p.8–11	Incorporation; new responsibility local / composite / new supplier	Actualization
`Actualization.pdf`	p.12–14	Point-of-Sale cashier-login change request; add & incorporate `Cashiers`	Actualization, JHotDraw Case Study
`Actualization.pdf`	p.15	Replacement of a class	Actualization

Source	Pages / slides	Topic(s) covered	Controlled-topic tags
`Actualization.pdf`	p.16–21	Change propagation walk (sale/register/store/item/taxCategory); termination	Actualization
`Actualization.pdf`	p.22	Deletion of obsolete functionality also propagates	Actualization
`Actualization.pdf`	p.23	Under-estimated impact set; propagation = moment of truth	Impact Analysis, Actualization
`Actualization.pdf`	p.24–25	Ericsson confusion matrix (136 classes); TP/FP/TN/FN	Impact Analysis
`Actualization.pdf`	p.26–27	Precision (100%) and Recall (32%)	Impact Analysis, Software Testing
`Actualization.pdf`	p.28	Under-estimation is systemic (invisibility)	Impact Analysis
`Clean Architecture.pdf [Martin]`	p.1–2	Title; architecture vs design pattern	Clean Architecture, Design Patterns
`Clean Architecture.pdf`	p.3–4	Four characteristics: testable, independent of UI/DB/frameworks	Clean Architecture
`Clean Architecture.pdf`	p.5–8	Concentric layers; layer definitions; Dependency Rule (inward)	Clean Architecture, OO Principles
`Clean Architecture.pdf`	p.9–14	Boundary/Interactor; Request/Response Models; full data-flow (Figs 1.1–1.6)	Clean Architecture
`Clean Architecture.pdf`	p.15	Response Model, Presenter, View Model, View defined	Clean Architecture
`Clean Architecture.pdf`	p.16–18	DB is a detail / plug-in; Entity Gateway/Repository (Fig 1.7)	Clean Architecture, Technical Debt
`Clean Architecture.pdf`	p.19–20	Assembled diagram (Fig 1.8); References	Clean Architecture
`OOPrinciples.pdf [Martin]` / `[Larman04]`	p.2–3	SOLID history (Martin 2000, Feathers acronym); 5 one-line defs	OO Principles, Clean Code
`OOPrinciples.pdf`	p.4–5	SRP + UserService/SecurityService example	OO Principles
`OOPrinciples.pdf`	p.6–7	OCP + Validator/LoanApprovalHandler example	OO Principles
`OOPrinciples.pdf`	p.8–9	LSP + Bird/Ostrich example	OO Principles
`OOPrinciples.pdf`	p.10–11	ISP + IUser/IRole/IUserRole example	OO Principles
`OOPrinciples.pdf`	p.12–13	DIP + Payments/PaymentMethod (constructor injection) example	OO Principles
`OOPrinciples.pdf`	p.14	Composite Reuse Principle (composition over inheritance)	OO Principles
`OOPrinciples.pdf`	p.15	Principle of Least Knowledge / Law of Demeter	OO Principles
`OOPrinciples.pdf`	p.16–18	GRASP defined; Larman reference; list of 9	OO Principles, Design Patterns

Source	Pages / slides	Topic(s) covered	Controlled-topic tags
`OOPrinciples.pdf`	p.19–27	The 9 GRASP patterns defined (Info Expert … Protected Variations)	OO Principles, Design Patterns
`OOPrinciples.pdf`	p.28–30	(blank trailing slides)	—
`ActualizationLab.pdf [ActLab]`	p.1	Lab: actualization defined; objectives; portfolio work (SOLID + Clean Arch on CASE study)	Actualization, Clean Architecture, OO Principles, JHotDraw Case Study

Deck reference lists (verbatim from the final slides)

Clean Architecture deck — "References" `(Clean Architecture p.20)` lists seven web sources (the deck is compiled from Robert C. Martin's talk and several Clean Architecture write-ups):

1. `https://www.youtube.com/watch?v=o_TH-Y78tt4` (Robert C. Martin's Clean Architecture talk)
2. `https://proandroiddev.com/clean-architecture-data-flow-dependency-rule-615ffdd79e29` (the data-flow vs. dependency-rule article behind slide 8's right-hand diagram)
3. `https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html` (Martin's original 2012 Clean Architecture blog post — the concentric-circle source)
4. `https://medium.freecodecamp.org/a-quick-introduction-to-clean-architecture-990c014448d2`
5. `https://medium.com/@pxpgraphics/clean-architecture-3fe6907e7441`
6. `https://hackernoon.com/golang-clean-archithecture-efd6d7c43047`
7. `https://rubygarage.org/blog/clean-android-architecture`

OOPrinciples deck — "Reference" `(OOPrinciples p.17)`: "Larman, C., *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*, 3rd Ed. Prentice-Hall, 2004." — the single cited source, sitting between the GRASP intro (p.16) and the pattern list (p.18), which confirms that the deck's GRASP block paraphrases Larman. (The SRP illustration on `(OOPrinciples p.4)` additionally carries an "enjoy algorithms" image credit.)

Actualization deck has no references slide; its provenance is the footer on every slide — "Software Engineering: The Current Practice Ch. 8" — plus the "© 2012 Václav Rajlich" credit on `(Actualization p.16)`, i.e. the deck *is* Rajlich Ch. 8 `[Raj13]`.